



Faculty of Engineering & Technology
Electrical & Computer Engineering Department
ENCS4370
Computer Architecture
First Semester (2025/2026)
Project 2
Design and Verification of a Simple Pipelined
RISC Processor in Verilog

Prepared by:

Student Name: Razan Shalabi

ID: 1230874

Student Name: Shatha Abualrob

ID: 1231279

Student Name: Lara Daifallah

ID: 1230239

Instructor: Dr. Ayman Hroub

Section: 1

Date: 2026/1/9

Abstract

This project presents the design and implementation of a simplified 32-bit predicated RISC processor using Verilog HDL, focusing on a pipelined architecture to enhance instruction throughput. The processor supports a set of R-Type, I-Type, and J-Type instructions with predicated execution, enabling conditional execution of instructions based on the content of a designated predicate register. The design incorporates a 32-register file, separate instruction and data memories, and an ALU capable of performing arithmetic and logical operations. Pipeline stages—including instruction fetch (IF), instruction decode (ID), execution (EX), memory access (MEM), and write-back (WB)—are implemented with intermediate pipeline registers to ensure proper data flow and control signal propagation. Immediate values are appropriately extended (zero or sign extension) based on instruction type, while multiplexers and a decode-and-multiplexing unit manage operand selection and pipeline control. The processor has been verified through simulation with a comprehensive testbench and multiple binary programs covering all instruction types and features. This project demonstrates a complete understanding of RISC architecture, pipelined datapath design, and hardware-level instruction execution, providing a robust framework for future enhancements such as branch prediction or hazard handling.

.

Table Of Contents

Abstract.....	2
List of Figures:	4
1. Design and Implementation.....	5
1.1. Processor Specifications and Overview	5
1.2. Instruction Format & RTL Operations.....	8
1.2.1. Instruction Formats and Interpretations.....	8
1.2.2. Instructions' Encoding	10
1.2.3. Instruction RTL Micro Operations	11
1.3. Description of stages & Datapath Components	13
1.4. Detailed description of entire Datapath connections	22
1.4.1. Instruction Fetch (IF) Stage:	22
1.4.2. Instruction Decode Stage (ID):	25
1.4.3. Execution Stage (EX)	33
1.4.4. Memory Access Stage (MEM):	35
1.4.5. Write Back Stage (WB):	37
2. Control Path & Control Signals	38
1.1 Control Signals Description.....	39
2.2. Control Signals Truth Table	41
2.3. Boolean Equations	42
Forwarding Unit Selection Logic	43
4. Simulation & Testing.....	45
• Correctness of Individual Components.....	45
• Comprehensive Test program in Assembly language and binary format:	49
❖ Assembly & Machine Code :	49
❖ The expected Results:	50
❖ Actual Final Results:.....	52
• Final Pipelied Processor Waveform :	53
❖ Complete Program Run from Cycle 1 to Cycle 30:.....	55
Team Work.....	86
Conclusion	87

List of Figures:

Figure 1: Instruction memory(ROM)	13
Figure 2: PC control unit.....	14
Figure 3: IF-ID Buffer	14
Figure 4: Splitter.....	15
Figure 5: Control Unit.....	15
Figure 6: Register File	16
Figure 7: Predicate Unit	16
Figure 8: Extender	16
Figure 9: ID-EX Buffer.....	17
Figure 10: Forwarding Unit	17
Figure 11: ALU (Arithmetic Logic Unit).....	18
Figure 12: EX-MEM Buffer.....	18
Figure 13: Data Memory (RAM).....	19
Figure 14: MEM-WB Buffer.....	19
Figure 15: Write Back Multiplexer.....	20
Figure 16: Stall Unit	20
Figure 17: Kill Unit	21
Figure 18: Piplined Processor Datapath.....	22
Figure 19: Instruction Fetch Stage (IF).....	22
Figure 20: PC Splitter	23
Figure 21: PC+1 ADDE.....	23
Figure 22: Predicate Unit Internally	23
Figure 23: Stall Mux.....	24
Figure 24: Kill Mux.....	24
Figure 25: Decode Stage (ID)	25
Figure 26: Splittig and Extending.....	26
Figure 27: Internal ID Muxes.....	27
Figure 28:Forwarding Network.....	28
Figure 29:Predicate logic	29
Figure 30:Control Signals Multiplexers in the Decode Stage	30
Figure 31: Forwarding Unit and RAW Hazard Management	30
Figure 32: Execution Stage (EX).....	33
Figure 33: Memory Access Stage (MEM)	36
Figure 34: Write Back Stage (WB)	37
Figure 35: ALU TB Simulation.....	45
Figure 36: Data Memory TB.....	46
Figure 37: Extender TB	47
Figure 38: Predicate Unit TB	48
Figure 39:Forwarding Unit TB.....	49
Figure 40:Testing - Actual Final Results	52
Figure 41: pipelined Processor Waveform.....	53

1. Design and Implementation

In this project, the objective is to design and verify a simplified 32-bit predicated RISC processor using Verilog HDL. The design process begins with a detailed analysis of the provided Instruction Set Architecture (ISA), including instruction formats, opcode encoding, register usage, and predicated execution behavior. Based on this analysis, the required functional units and control logic are identified, enabling the construction of a coherent datapath and control path that support all specified instructions.

This report presents the adopted design methodology, describes the internal architecture of the processor, and explains how instructions are executed across the processor **pipeline stages**. Special attention is given to the implementation of predicated execution, which is a core feature of the processor. In this design, instructions pass through the pipeline stages simultaneously, allowing multiple instructions to be processed at the same time, thereby improving throughput.

1.1. Processor Specifications and Overview

The designed processor adheres strictly to the given project specifications, which are summarized as follows:

1. Instruction and Word Size

The processor operates on a **32-bit architecture**, where both the instruction size and the word size are fixed at **32 bits**.

2. Register File

The processor contains 32 general-purpose registers (R0 to R31), each 32 bits wide.

- R0 is hardwired to zero and cannot be modified.
- R30 is dedicated as the Program Counter (PC).
- R31 is used as the return address register, primarily for function calls.

3. Program Counter (PC)

The PC holds the address of the current instruction and is updated sequentially or through control-flow instructions such as jumps and procedure calls. Pipeline registers ensure that each instruction has access to the correct PC value.

4. Memory Architecture

The processor employs a Harvard architecture, consisting of:

- A dedicated instruction memory
- A separate data memory

Both memories are word-addressable, ensuring efficient access to aligned 32-bit data. The pipeline allows simultaneous instruction fetch and data memory access.

5. Instruction Types

The ISA supports three instruction formats:

- **R-Type (Register Type)** for register-to-register operations
- **I-Type (Immediate Type)** for operations involving immediate values or memory access
- **J-Type (Jump Type)** for control flow instructions

6. Predicated Execution

Each instruction includes a predicate register field (Rp), which determines whether the instruction is executed:

- If $\text{Reg}[\text{Rp}] \neq 0$, the instruction executes normally.
- If $\text{Reg}[\text{Rp}] = 0$, the instruction passes through all pipeline stages but has no effect on the processor state.
- Unconditional execution is achieved by setting $\text{Rp} = \text{R0}$.

This mechanism reduces control hazards and minimizes explicit branching, reducing control hazards in the pipeline.

7. Immediate and Offset Handling

- Immediate values are **zero-extended** for logical instructions (ANDI, ORI, NORI).
- Immediate values and jump offsets are **sign-extended** for arithmetic and control instructions (ADDI, LW, SW, J, CALL).

- All negative values are represented using two's complement

8. Pipeline Overview

The processor implements a **five-stage pipeline**, with each instruction passing through the following stages:

- **Instruction Fetch (IF)** – Fetch instruction from instruction memory
- **Instruction Decode (ID)** – Decode instruction and read operands
- **Execute (EX)** – Perform ALU operations or calculate addresses
- **Memory Access (MEM)** – Read from or write to data memory
- **Write-Back (WB)** – Update registers with results

Pipeline registers between stages (IF/ID, ID/EX, EX/MEM, MEM/WB) store instruction fields, operands, and intermediate results, enabling multiple instructions to progress through the pipeline simultaneously. Hazards (data, control, structural) are handled using forwarding, stalling, and predicated execution mechanisms.

1.2. Instruction Format & RTL Operations

1.2.1. Instruction Formats and Interpretations

The Instruction Set Architecture (ISA) of the proposed processor defines **three instruction formats**, all instructions are **32 bits wide** and include a **predicate register field (Rp)** that enables predicated execution. Instructions flow through the pipeline stages $IF \rightarrow ID \rightarrow EX \rightarrow MEM \rightarrow WB$. Operands are read in ID, ALU operations occur in EX, memory accesses in MEM, and register updates in WB. The supported instruction formats are described as follows:

1. R-Type (Register Type)

This format is used for register-based operations and is structured as follows:

Opcode ⁵	Rp ⁵	Rd ⁵	Rs ⁵	Rt ⁵	Unused ⁷
---------------------	-----------------	-----------------	-----------------	-----------------	---------------------

Table 1: R_Type Instruction Format

- **5-bit Opcode:** Specifies the operation to be performed
- **5-bit Rp :** Determines whether the instruction is executed:

If $\text{Reg}[\text{Rp}] \neq 0$, the instruction executes.

If $\text{Reg}[\text{Rp}] = 0$, the instruction has no effect.

- **5-bit Rd:** Destination register.
- **3-bit Rs:** First source register.
- **5-bit Rt:** Second source register.
- **7-bit Unused.**

2. I-Type (Immediate Type)

This format is designed for instructions involving immediate values or memory access and follows this structure:

Opcode ⁵	Rp ⁵	Rd ⁵	Rs ⁵	Imm ¹²
---------------------	-----------------	-----------------	-----------------	-------------------

Table 2: I_Type Instruction Format

- **5-bit Opcode:** Specifies the instruction type.
- **5-bit Rp:** Controls conditional execution of the instruction.
- **5-bit Rd:** Destination register.
- **5-bit Rs:** Source register.
- **12-bit Immediate:** Used as an immediate operand or memory offset:
 - **Zero-extended** for logical instructions (ANDI, ORI, NORI)
 - **Sign-extended** for arithmetic and memory instructions (ADDI, LW, SW)

For memory access instructions, the effective address is computed as:

$$\text{Effective Address} = \text{Reg}[\text{Rs}] + \text{Sign-Extended Immediate}$$

3. J-Type (Jump Type)

The **J-Type format** is reserved for control-flow instructions such as unconditional jumps and procedure calls. Its format is shown below:

Opcode ⁵	Rp ⁵	Offset ²²
---------------------	-----------------	----------------------

Table 3: J_Type Instruction Format

- **5-bit Opcode:** Specifies the jump function (J or Call).
- **5-bit Rp:** Determines whether the jump is taken.
- **22-bit Offset:** Represents a signed displacement added to the current PC.
- **Jump Target Address Calculation:**

$$\text{Jump Target} = \text{PC} + \text{Sign-Extended Offset}$$

For the CALL instruction, the return address for example the address of the next instruction is stored in register **R31** before updating the PC.

These three instruction formats collectively enable the execution of arithmetic, logical, memory access, and control-flow operations while supporting predicated execution in the pipeline.

1.2.2. Instructions' Encoding

The processor implements a subset of the ISA covering essential operations. Each instruction is uniquely encoded using an opcode value. Unlike traditional designs that use separate function fields, opcode-based decoding is used, simplifying control logic. Instructions flow through the pipeline stages, and predication controls whether writes to registers or memory occur in WB or MEM stages.

Instruction	Type	Meaning	Opcode
ADD Rd, Rs, Rt, Rp	R-Type	If (Reg[Rp] $\neq$ 0) Reg[Rd] = Reg[Rs] + Reg[Rt]	0
SUB Rd, Rs, Rt, Rp	R-Type	If (Reg[Rp] $\neq$ 0) Reg[Rd] = Reg[Rs] - Reg[Rt]	1
OR Rd, Rs, Rt, Rp	R-Type	If (Reg[Rp] $\neq$ 0) Reg[Rd] = Reg[Rs] Reg[Rt]	2
NOR Rd, Rs, Rt, Rp	R-Type	If (Reg[Rp] $\neq$ 0) Reg[Rd] = $\sim$ (Reg[Rs] Reg[Rt])	3
AND Rd, Rs, Rt, Rp	R-Type	If (Reg[Rp] $\neq$ 0) Reg[Rd] = Reg[Rs] & Reg[Rt]	4
ADDI Rd, Rs, Imm, Rp	I-Type	If (Reg[Rp] $\neq$ 0) Reg[Rd] = Reg[Rs] + Imm	5
ORI Rd, Rs, Imm, Rp	I-Type	If (Reg[Rp] $\neq$ 0) Reg[Rd] = Reg[Rs] Imm	6
NORI Rd, Rs, Imm, Rp	I-Type	If (Reg[Rp] $\neq$ 0) Reg[Rd] = $\sim$ (Reg[Rs] Imm)	7
ANDI Rd, Rs, Imm, Rp	I-Type	If (Reg[Rp] $\neq$ 0) Reg[Rd] = Reg[Rs] & Imm	8
LW Rd, Imm(Rs), Rp	I-Type	If (Reg[Rp] $\neq$ 0) Reg[Rd] = Mem(Reg[Rs] + Imm)	9
SW Rd, Imm(Rs), Rp	I-Type	If (Reg[Rp] $\neq$ 0) Mem(Reg[Rs] + Imm) = Reg[Rd]	10
J Label, Rp	J-Type	If (Reg[Rp] $\neq$ 0), jump to the target address	11
CALL Label, Rp	J-Type	If (Reg[Rp] $\neq$ 0) , jump to the function, store the return address in register R31	12
JR, Rs, Rp	R-Type	If (Reg[Rp] $\neq$ 0) , jump to the address in register Rs	13

Table 4: Instruction's Encoding Table

1.2.3. Instruction RTL Micro Operations

In this section, we define the Register Transfer Language (RTL) micro-operations for the implemented instructions in the context of a **pipelined processor**. Each instruction passes through five pipeline stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB). **Predicated execution** is enforced in the EX, MEM, or WB stages, depending on the instruction type. Even if the predicate register $R_p = 0$, instructions pass through all pipeline stages to maintain timing and pipeline integrity.

Intermediate values are stored in **pipeline registers** between stages (IF/ID, ID/EX, EX/MEM, MEM/WB) to allow simultaneous instruction execution.

Group 1: R-Type (Register Operations): *IF → ID → EX → WB*

1. **Fetch instruction:** $IR \leftarrow Mem[PC]$
2. **Decode instruction:**
 $data1 \leftarrow Reg[Rs]$
 $data2 \leftarrow Reg[Rt]$
 $predicate \leftarrow Reg[Rp]$
3. **Execute :** $ALU_result \leftarrow ALU_function(data1, data2)$
4. **Write back to Rd :** $If\ predicate \neq 0 \rightarrow Reg[Rd] \leftarrow ALU_result$
5. **Update PC:** $PC \leftarrow PC + 1$

Group 2: I-Type (Arithmetic/ Logical with imm): *IF → ID → EX → WB*

1. **Fetch Instruction:** $IR \leftarrow Mem[PC]$
2. **Decode instruction:**
 $data1 \leftarrow Reg[Rs]$
 $imm \leftarrow \text{Sign-Extended or Zero-Extended Immediate}$
 $predicate \leftarrow Reg[Rp]$
3. **Execute:** $ALU_result \leftarrow ALU_opcode(data1, imm)$
4. **Write back to Rd :** $If\ predicate \neq 0 \rightarrow Reg[Rt] \leftarrow ALU_result$
5. **Update PC:** $PC \leftarrow PC + 1$

** Signed Extension at ADDI , Unsigned Extension at ANDI*

Group 3: I-Type (Load Word):***IF → ID → EX → M → WB***

1. Fetch instruction:
2. Decode instruction:
3. Execute:
4. M (Read from memory):
5. Write back to Rd:
6. Update PC:

$IR \leftarrow Mem[PC]$
 $base \leftarrow Reg[Rs]$
 $predicate \leftarrow Reg[Rp]$
 $Address \leftarrow base + Sign_Extended\ Imm$
If $predicate \neq 0 \rightarrow data \leftarrow Mem[Address]$
If $predicate \neq 0 \rightarrow Reg[Rd] \leftarrow data$
 $PC \leftarrow PC + 1$

Group 4: I-Type (Store Word):***IF → ID → EX → M***

1. Fetch instruction:
2. Decode instruction:
3. Execute:
4. M (Read from memory):
5. Update PC:

$IR \leftarrow Mem[PC]$
 $base \leftarrow Reg[Rs]$
 $data \leftarrow Reg[Rt]$
 $predicate \leftarrow Reg[Rp]$
 $Address \leftarrow base + Sign_Extended\ Imm$
If $predicate \neq 0 \rightarrow Mem[Address] \leftarrow data$
 $PC \leftarrow PC + 1$

Group 5: J-Type (Jump):***IF → ID***

1. Fetch Instruction:
2. Decode instruction:

$IR \leftarrow Mem[PC]$
 $target_address \leftarrow PC + Sign_Extended\ Offset + 1$
if $predicate \neq 0 \rightarrow PC \leftarrow target_address$ else $PC \leftarrow PC + 1$

Group 6: J-Type (Call):***IF → ID → WB***

1. Fetch Instruction:
2. Decode instruction:
Calculate Target address
& Save Return Instruction Address
3. WB

$IR \leftarrow Mem[PC]$
if $predicate \neq 0 \rightarrow target_address \leftarrow PC + Sign\ Ext\ Offset + 1$
 $PC \leftarrow target_address$
if $predicate \neq 0 \rightarrow R31 \leftarrow PC + 1$ else $PC \leftarrow PC + 1$

Group 7: J-Type (Return):***IF → ID***

1. Fetch Instruction:
2. Decode instruction:

$IR \leftarrow Mem[PC]$
if $predicate \neq 0 \rightarrow PC \leftarrow R[31]$ else $PC \leftarrow PC + 1$

1.3. Description of stages & Datapath Components

In the design of our pipelined 32 bit processor, several functional units and components are essential for ensuring the correct execution of instructions. These units work together to process data, perform arithmetic and logical operations, handle memory access, and manage control flow in a pipelined manner. The processor is implemented as a 5-stage pipelined RISC architecture consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB). A unique feature of this datapath is its predicated execution logic, which gates the control signals based on the value of a predicate register (Rp) . Furthermore, the Program Counter (PC) is physically mapped to Register 30 (R30) within the Register File, and the Return Address is hardwired to Register 31 (R31).

Those five stages are as follows:

1. Instruction Fetch Stage (IF)

The Instruction Fetch (IF) stage is responsible for obtaining the next instruction from the Instruction Memory, and it contains the following components:

- **Program Counter (R30)**

Unlike conventional RISC processors where the Program Counter (PC) is implemented as a dedicated standalone register, this design stores the PC inside the Register File as register **R30**. During execution, the Instruction Fetch stage receives the updated PC value (next_pc) from the PC Control Unit, which is then written into R30 at the rising edge of the clock.

- **Instruction Memory:**

The Instruction Memory is a word-addressable read-only memory (ROM) that stores the program instructions. It uses the current PC value as its address input and outputs a 32-bit instruction corresponding to that address. The instruction fetch operation is performed combinatorially.

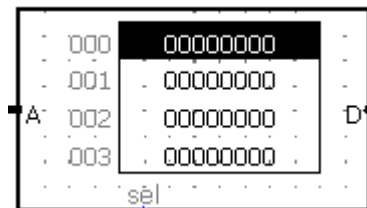


Figure 1: Instruction memory(ROM)

- **PC Control Unit:**

The PC Control Unit is responsible for determining the next value of the Program Counter. Based on the PCSrc control signal, it selects one of the following:

- **PC + 1** for normal sequential execution.
- **PC + Offset** for Jump and Call instructions.
- **Register value** for Jump Register (JR) instructions.

This decision is gated by the ExecuteEn signal generated in the Decode stage. If the predicate evaluates to false, jump-related updates are ignored and the PC advances sequentially

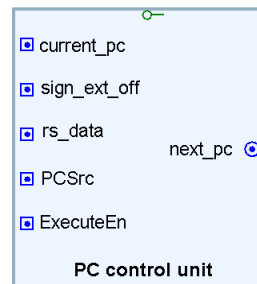


Figure 2: PC control unit

- **IF/ID Pipeline Buffer:**

The IF/ID pipeline buffer buffers the fetched instruction and the incremented PC value (**PC + 1**) and forwards them to the Decode stage. It supports:

- A **Stall** signal, which freezes the register contents during load-use hazards.
- A **Kill** signal, which flushes the instruction by replacing it with a NOP in case of a taken jump or call.

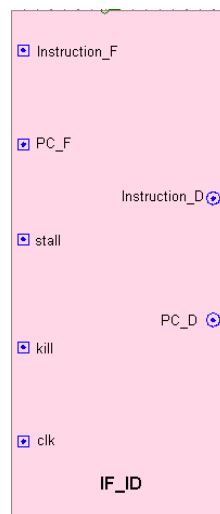


Figure 3: IF-ID Buffer

2. Instruction Decode Stage (ID):

The Instruction Decode stage acts as the control and hazard-management center of the processor, and it contains the following components:

- **Splitter:**

The Splitter is a combinational unit that decomposes the 32-bit instruction into its individual fields: Opcode, predicate register (Rp), destination register (Rd), source registers (Rs and Rt), immediate value, and jump offset.



Figure 4: Splitter

- **Control Unit:**

The Control Unit decodes the Opcode field and generates all necessary control signals, including RegWrite, MemRead, MemWrite, ALU_Op, WB_Sel, PCSrc, and ExtSel. These signals guide data movement and operation selection throughout the datapath.

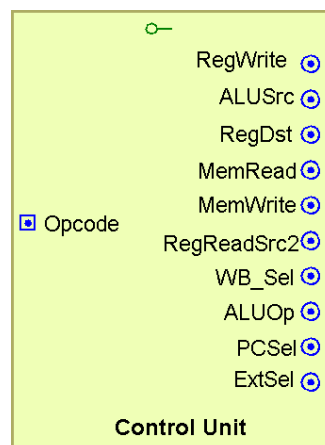


Figure 5: Control Unit

- **Register File (RegFile):**

The Register File consists of 32 registers, each 32 bits wide. It provides three read ports for Rs, Rt, and the predicate register Rp, and one write port used during the Write-Back stage. Register R0 is permanently hardwired to zero, while register R30 holds the Program Counter and is updated every clock cycle by the IF stage.

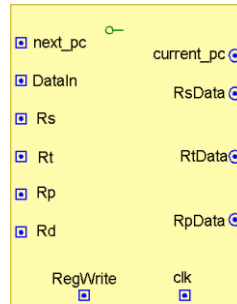


Figure 6: Register File

- **Predicate Unit:**

The Predicate Unit implements predicated execution. It examines the value stored in the selected predicate register (Rp). If $R_p \neq 0$, or if Rp is R0, the unit asserts the ExecuteEn signal. If $R_p = 0$, the instruction is effectively converted into a NOP by disabling register and memory write operations.

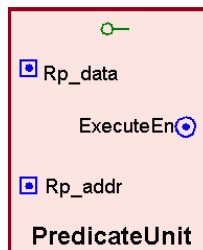


Figure 7: Predicate Unit

- **Extender:**

The Extender unit generates 32-bit immediate values from instruction fields. Based on the ExtSel control signal, it performs zero-extension for logical instructions or sign-extension for arithmetic and control instructions, extending 12-bit immediates or 22-bit offsets.

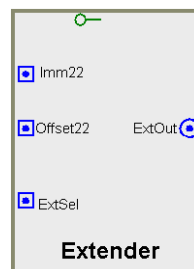


Figure 8: Extender

- **ID/EX Pipeline Buffer:**

The ID/EX pipeline buffer transfers operand values and control signals from the Decode stage to the Execution stage. If a stall is detected, this register is cleared to insert a pipeline bubble.

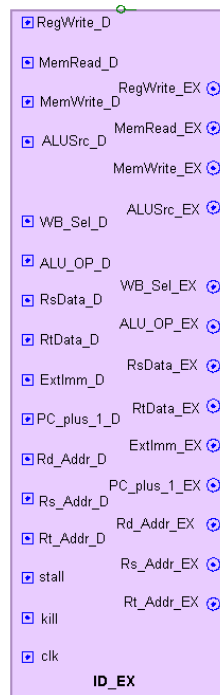


Figure 9: ID-EX Buffer

- **Forwarding Unit:**

To minimize pipeline stalls, the Forwarding Unit resolves data hazards by monitoring destination registers in the EX, MEM, and WB stages. When a source register matches a destination register in a later stage, the most recent value is forwarded directly to the ALU, bypassing the Register File.

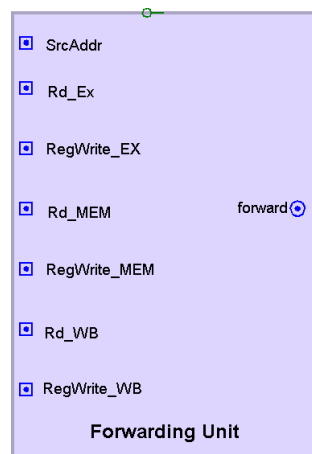


Figure 10: Forwarding Unit

3. Execution Stage (EX):

The Execution stage performs arithmetic and logical computations. And it contains the following components:

- **Arithmetic Logic Unit (ALU):**

The ALU executes operations such as ADD, SUB, AND, OR, and NOR. Its inputs are selected through multiplexers that choose between register operands (including forwarded values from later pipeline stages) and extended immediate values.

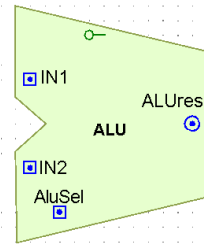


Figure 11: ALU (Arithmetic Logic Unit)

- **EX/MEM Pipeline Buffer:**

The EX/MEM Buffer buffers the ALU result (used as a memory address) and the data to be written to memory, ensuring proper synchronization between stages.

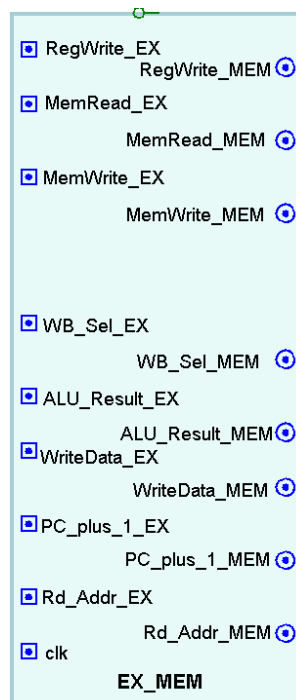


Figure 12: EX-MEM Buffer

4. Memory Access Stage (MEM):

This stage interfaces with the processor's data storage, and it contains the following

components:

- **Data Memory (RAM):**

The Data Memory is a 4 KB word-addressable memory. For Load Word (LW) instructions, it reads data from the address computed by the ALU. For Store Word (SW) instructions, it writes data from Rd register into memory on the rising edge of the clock.

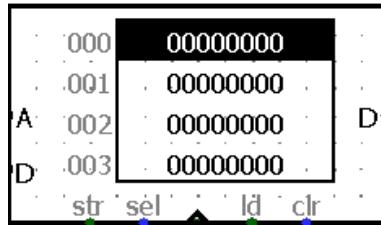


Figure 13: Data Memory (RAM)

- **MEM/WB Pipeline Buffer:**

This pipeline register holds the selected write-back data and destination register address until the final write operation occurs.

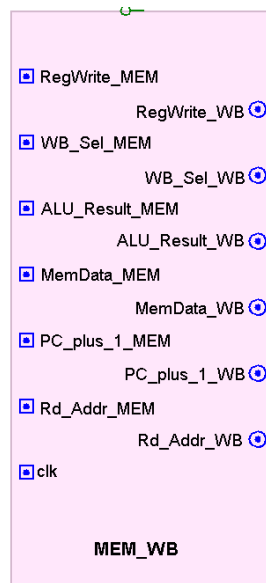


Figure 14: MEM-WB Buffer

5. **Write Back Stage (RB):**

The Write-Back stage completes instruction execution by updating the Register File, and it contains the following components:

- **Write-Back Multiplexer:**

A 3-to-1 multiplexer selects the value to be written back to the destination register (Rd).
The selected value can be:

- The ALU result (for arithmetic and logical instructions),
- Data read from memory (for load instructions),
- PC + 1 (for CALL instructions to save the return address in register R31).

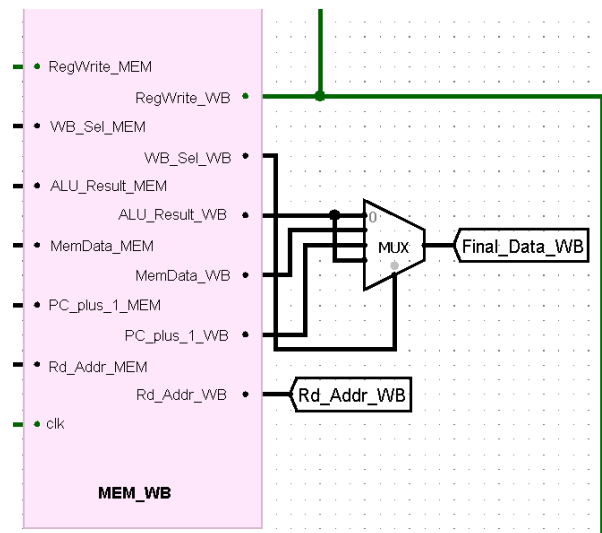


Figure 15: Write Back Multiplexer

6. **Hazard Management Units:**

- **Stall Unit:**

The Stall Unit detects load-use hazards. If an instruction in the Decode stage depends on data being loaded by an instruction currently in the Execution stage, the Stall Unit pauses the IF and ID stages for one cycle to preserve correctness.

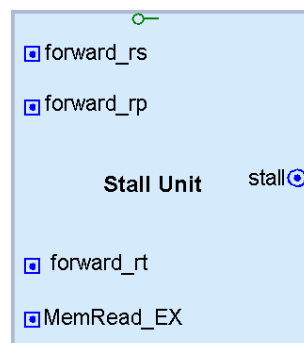


Figure 16: Stall Unit

- **Kill Unit:**

The Kill Unit resolves control hazards. When a Jump, Call, or JR instruction is confirmed to execute (based on the predicate evaluation), it flushes the instruction currently in the IF stage to prevent incorrect execution along the wrong control path.

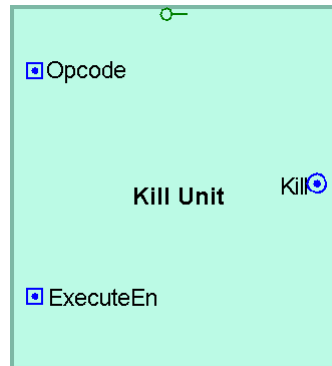


Figure 17: Kill Unit

1.4. Detailed description of entire Datapath connections

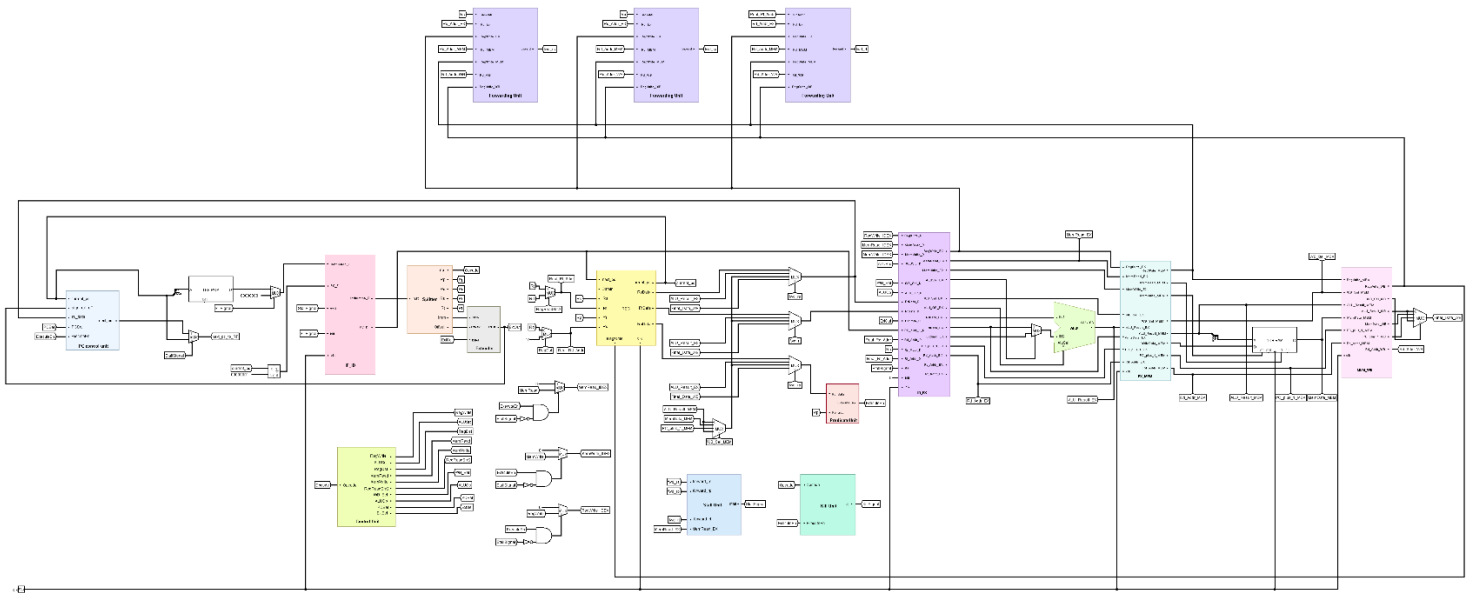


Figure 18: Piplined Processor Datapath

As mentioned before, our processor implements a five stage pipelined datapath. The datapath is organized so that each pipeline stage operates concurrently on a different instruction, with pipeline buffers isolating stages and control logic coordinating data movement between them. Now lets go with each stage:

1.4.1. Instruction Fetch (IF) Stage:

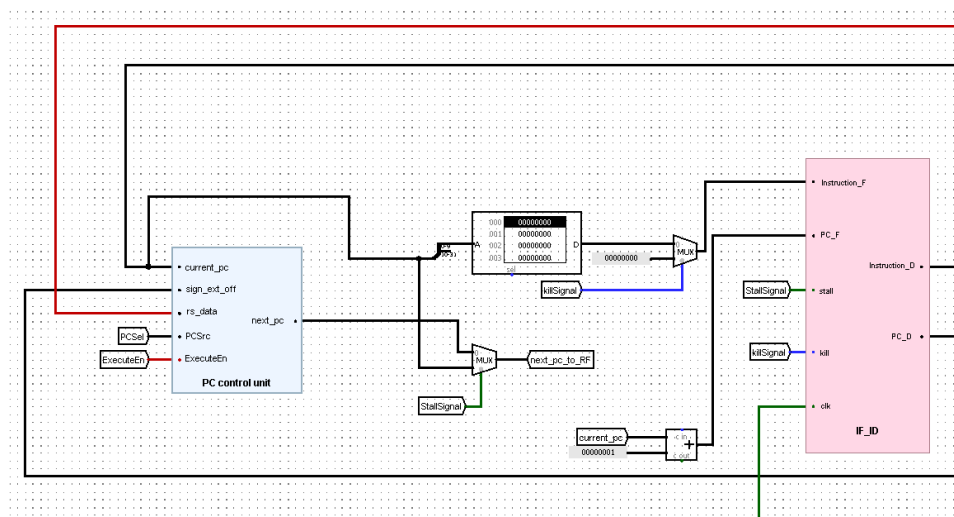


Figure 19: Instruction Fetch Stage (IF)

The Instruction Fetch (IF) stage forms the entry point of the pipeline and is responsible for determining which instruction enters the pipeline next. In this stage, the PC Control Unit calculates the next_pc value based on the current program flow. This value is written back to the Register File in register R30, which holds the Program Counter, and is at the same time used as the address input to the Instruction Memory, which is a 1 KB word addressable ROM that outputs a 32 bit instruction.

Note: A splitter is used to extract only the lower bits of the PC before accessing the Instruction Memory. Since the instruction memory is a small 1 KB word addressable ROM, it does not require the full 32 bit PC to index its entries. The lower 10 bits of the PC are sufficient to uniquely address all instruction locations within the memory.

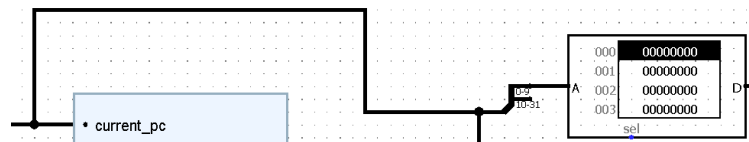


Figure 20: PC Splitter

The current PC value is read from register R30 and routed to both the Instruction Memory (as in figure 20) and a PC + 1 adder (figure 21). The PC Control Unit receives several inputs, including the current PC value, the sign-extended offset from the Decode stage, register data for Jump Register (JR) instructions, the PCSrc control signal from the Control Unit, and the ExecuteEn signal from the Predicate Unit. A multiplexer inside the PC Control Unit selects between PC + 1, PC + Offset for Jump and Call instructions, or the register value for JR instructions. This selection is handled by the ExecuteEn signal to ensure that predicated-false control instructions do not modify the program flow, preserving correct execution under predicated RISC semantics.

While $\text{ExecuteEn} = (\text{Rp} = 0) \text{ OR } (\text{Reg}[\text{Rp}] \neq 0)$

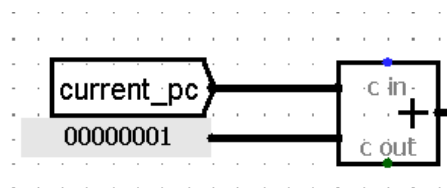


Figure 21: PC+1 ADDE

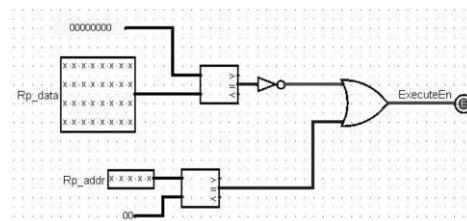


Figure 22: Predicate Unit Internally

To handle pipeline hazards, the IF stage handles both stall and kill mechanisms. A stall-controlled multiplexer feeds the next_pc_to_RF signal. When the StallSignal is asserted, the PC is not updated, and so freezing the IF stage and allowing coming instructions to wait until the required data becomes available. This prevents incorrect instruction improvement during load use hazards.

Control hazards are handled using a NOP injection mechanism. A 2-to-1 multiplexer, is placed after the Instruction Memory and is controlled by the killSignal generated in the Decode stage. When a Jump, Call, or JR instruction is taken, the Kill Unit asserts killSignal, causing the multiplexer to replace the fetched instruction with a NOP (0x00000000) before it enters the IF/ID pipeline register. This prevents wrong instructions from being maintained

Overall, the IF stage ensures correct instruction sequencing, supports predicated control flow, and integrates stall and kill logic to maintain pipeline correctness.

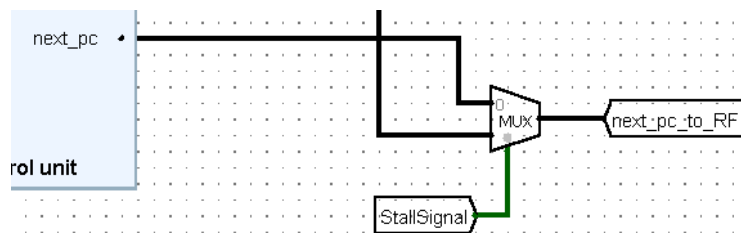


Figure 23: Stall Mux

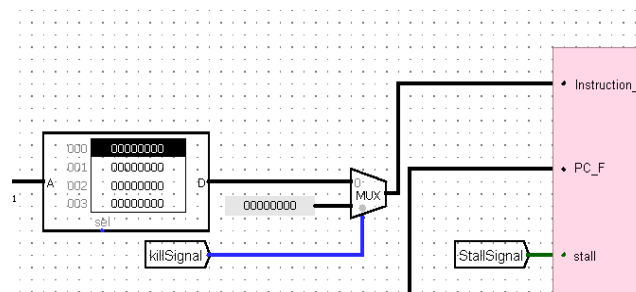


Figure 24: Kill Mux

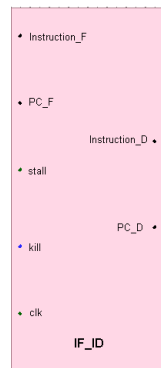
- **IF-ID Pipeline Buffer :**

The IF/ID pipeline buffer acts as the boundary between the Instruction Fetch (IF) stage and the Instruction Decode (ID) stage. Its main role is to temporarily store the fetched instruction and the incremented program counter value (PC + 1) before they are passed to the Decode stage in the next clock cycle.

During normal operation, the IF/ID register captures the fetched instruction (Instruction_F) and the PC + 1 value (PC_F) on every clock edge. If a stall is asserted, the register holds its current contents, preventing a new instruction from entering into the Decode stage. This ensures that

instructions don't move forward before required data becomes available. If a control hazard occurs due to a taken Jump, Call, or JR instruction, a kill signal is asserted. In this case, the IF/ID register replaces the fetched instruction with a NOP.

It takes the inst_F and PC_F and StallSignal and KillSignal as inputs (and for sure clk) and outputs Inst_D and PC_D as entries for decode stage .



1.4.2. Instruction Decode Stage (ID):

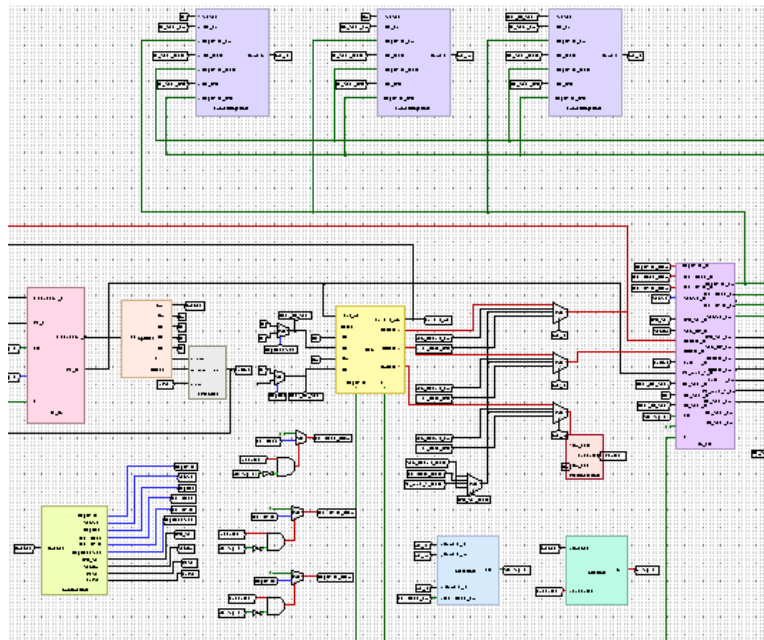


Figure 25: Decode Stage (ID)

The Instruction Decode (ID) stage is the most complex stage of the datapath, as it is responsible for decoding instructions, accessing registers, generating control signals, and managing both data and control hazards.

During instruction parsing, the fetched 32 bit instruction is processed by the Splitter, which separates it into its individual fields, including the opcode, predicate register (Rp), destination register (Rd), source registers (Rs and Rt), immediate value, and jump offset. At the same time, the Control Unit decodes the opcode and generates the required control signals such as RegWrite, MemRead, MemWrite, ALU_Op, WB_Sel, PCSrc, and ExtSel. In parallel, the Extender generates a 32 bit immediate value by either zero-extending or sign-extending the instruction fields depending on the instruction type as shown in figure 26. Furthermore, the ID stage performs the jump target calculation by adding the sign-extended offset to the current PC value passed from the IF/ID buffer.

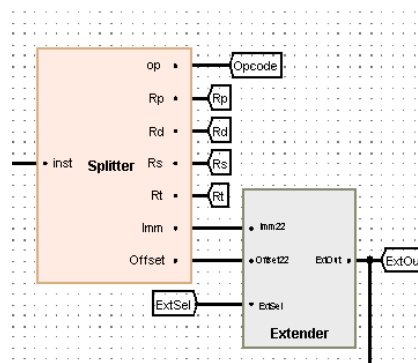


Figure 26: Splitting and Extending

The ID stage also performs register access through the 32 entry, 32 bit Register File. It supports three read ports for Rs, Rt, and the predicate register Rp, as well as one write port used during the Write Back stage. Internal multiplexers are used to support special ISA behaviors. Because the instruction set supports different instruction formats and special cases such as STORE and CALL, multiplexers are required to dynamically choose which register fields are used during execution.

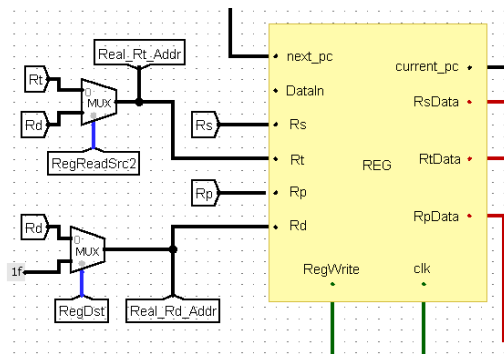


Figure 27: Internal ID Muxes

The RegReadSrc2 multiplexer selects the source register address for the second read port of the Register File. For most instructions, this input comes from the Rt field. However, for store instructions (SW), the data to be written to memory is located in the Rd field instead. The RegReadSrc2 control signal selects between Rt and Rd, producing the Real_Rt_Addr signal. This allows the same Register File hardware to correctly support multiple instruction formats .

The RegDst multiplexer determines the destination register address used for write back. For normal operation, the destination register is taken from the Rd field. For CALL instructions, however, the return address must be written to register R31 (1f). When RegDst is asserted, the multiplexer forces the destination address to R31, producing the Real_Rd_Addr signal. This enables CALL instructions to store the return address without requiring special purpose hardware.

The Register File itself provides three read outputs: RsData, RtData, and RpData, corresponding to the source registers Rs, Rt, and the predicate register Rp. It also receives the next_pc value to update register R30, which holds the Program Counter. The RegWrite signal enables writing to the selected destination register on the rising edge of the clock, completing the write back operation.

Now, as we mentioned before, The Register File outputs the values of RsData, RtData, and RpData, these values may be outdated if a previous instruction has not yet written its result back. To handle this, each of these outputs is connected to a multiplexer that can select the most recent value from later pipeline stages.

For each source operand (Rs, Rt, and Rp), a forwarding multiplexer selects between the value read from the Register File, the ALU result from the Execute stage, or the final write-back value from

the MEM/WB stage. The selection is controlled by forwarding control signals (fwd_rs, fwd_rt, and fwd_rp) generated by the Forwarding Unit. This allows dependent instructions to immediately use results produced by earlier instructions without waiting for them to be written back to the Register File.

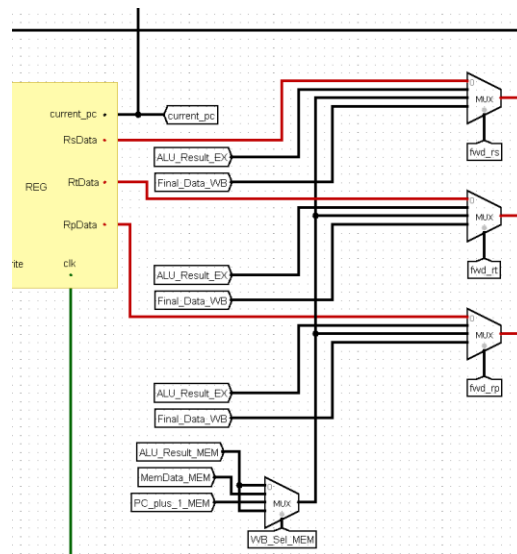


Figure 28: Forwarding Network

The forwarding logic is also applied to the predicate register path (RpData). This ensures that predicate evaluation always uses the last updated value, which ensures correct predicated execution. Without forwarding on the predicate path, the processor could incorrectly enable or disable instructions based on stale predicate values.

The forwarding network also includes a specific WB_Sel_MEM multiplexer (visible in the forwarding logic). This sub-selection is necessary because the MEM stage might contain an ALU result (for arithmetic instructions), data recently read from memory (for load instructions), or a return link (for CALL instructions). The WB_Sel_MEM control signal ensures that the correct result is chosen for forwarding to the Rs, Rt, and Rp multiplexers. This allows the processor to resolve data hazards even when the preceding instruction is a memory load or a procedure call.

The Predicate Unit (figure 29) receives forwarded data from the EX, MEM, and WB stages through dedicated forwarding paths. The ExecuteEn signal is then used to gate the RegWrite, MemRead, and MemWrite control signals, effectively converting instructions with a false predicate into NOPs without disrupting pipeline flow.

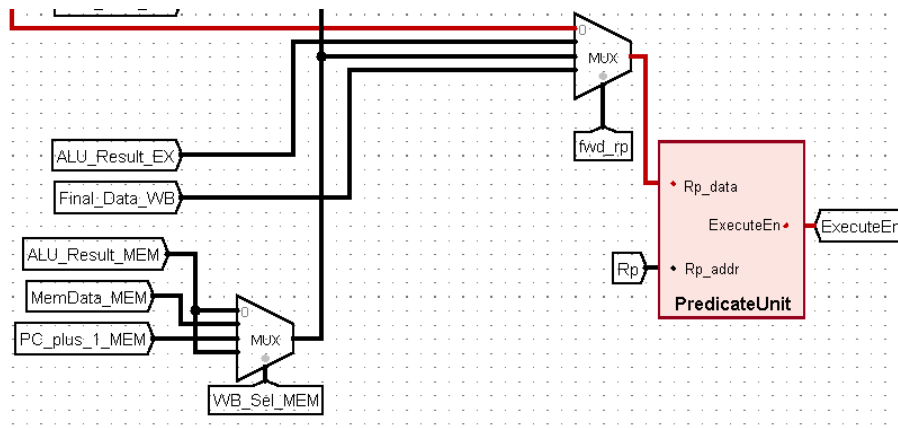


Figure 29: Predicate logic

Hazard detection is also handled within the ID stage. The Stall Unit detects load-use hazards by monitoring instructions in the EX stage and freezes the IF and ID stages when necessary to maintain data integrity. Additionally, the Kill Unit resolves control hazards by flushing the Fetch stage whenever a Jump, Call, or JR instruction is confirmed to execute based on the predicate value. This creates a vital feedback loop between the two stages: the target address, the PCSrc selection, and the ExecuteEn signal are sent backward from the Decode stage to the PC Control Unit in the Fetch stage. By resolving control-flow decisions in the Decode stage, the processor only needs to flush a single instruction, resulting in a one cycle penalty. This significantly improves performance compared to designs that resolve branches in the Execute stage.

The multiplexers shown in figure 30 are used to control how the main control signals are transferred into the ID/EX pipeline register. Each multiplexer corresponds to one critical control signal: MemRead, MemWrite, and RegWrite. The role of these multiplexers is to decide whether the original control signal should proceed normally or be forced to zero.

Each multiplexer has two inputs: the original control signal generated by the Control Unit and a constant zero value. The selection signal for each multiplexer is produced by combining the ExecuteEn signal with the inverted StallSignal. When the instruction is allowed to execute and no stall is present, the multiplexer selects the original control signal and forwards it to the ID/EX register. This allows the instruction to perform its intended operation in later pipeline stages.

If either the predicate condition is false (ExecuteEn = 0) or a stall is asserted, the multiplexer selects zero instead. This forces the corresponding control signal entering the Execute stage to be

disabled. As a result, the instruction behaves like a NOP, preventing any register write or memory access while pipeline works as intended.

These multiplexers play a critical role in pipelined execution by safely inserting bubbles during stalls and supporting predicated execution without flushing the pipeline.

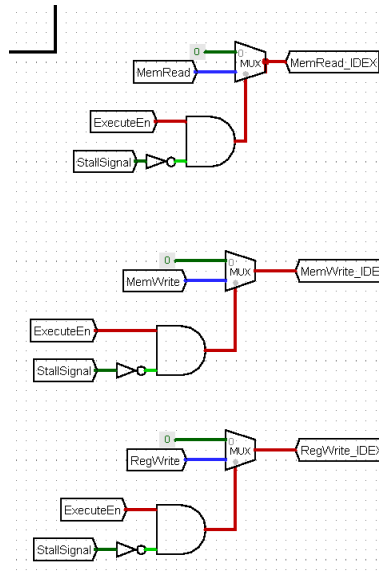


Figure 30: Control Signals Multiplexers in the Decode Stage

- **Forwarding Unit and RAW Hazard Management**

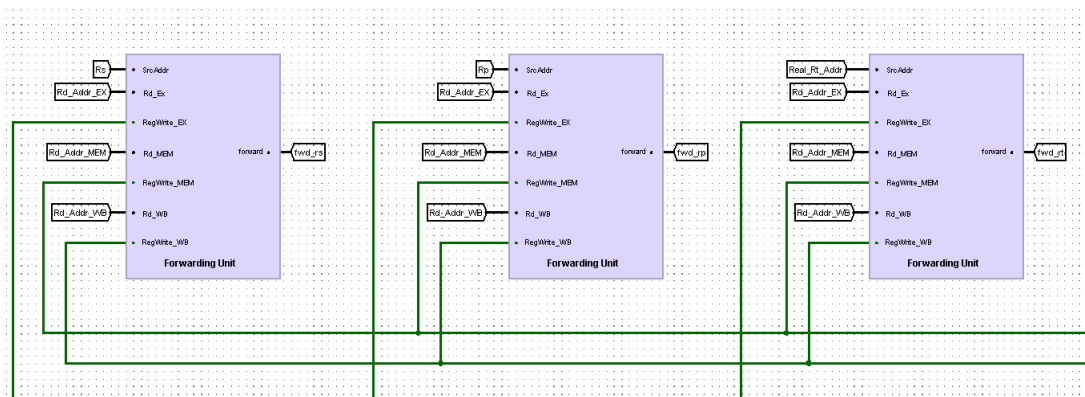


Figure 31: Forwarding Unit and RAW Hazard Management

A. Forwarding Unit Logic

Our datapath contains three independent Forwarding Units, one for each register read path: Rs, Rt, and the predicate register Rp. Each unit continuously monitors the pipeline to detect data dependencies.

Each Forwarding Unit receives the source register address of the current instruction and compares it against the destination register addresses of instructions currently in the Execute, Memory, and Write-Back stages. These addresses are labeled Rd_EX, Rd_MEM, and Rd_WB, respectively, along with their corresponding RegWrite control signals.

To ensure that the most recent value is always selected, the Forwarding Unit applies a strict priority scheme. If the source register matches Rd_EX and RegWrite_EX is asserted, the ALU result from the Execute stage is selected, as this represents the most recent value. If no match is found in the Execute stage, the unit checks the Memory stage. If SrcAddr matches Rd_MEM, the value from the MEM stage is selected. If neither of these conditions is met, the Write-Back stage is checked. If no dependency is detected in any stage, the value read directly from the Register File is used.

B. Predicate (Rp) Forwarding

Unlike conventional RISC pipelines, which only forward values for Rs and Rt into the Execute stage, this design also forwards the predicate register Rp directly into the Decode stage. This is a deliberate architectural choice.

Because predicate evaluation and control-flow decisions occur in the Decode stage, the Predicate Unit and PC Control Unit must always operate on the most recent value of Rp. By forwarding Rp from the Execute, Memory, or Write-Back stages, the processor ensures that predicated instructions and control-flow operations such as Jump and CALL are evaluated correctly without introducing additional stalls.

C. MEM-Stage Selection Multiplexer

The Memory stage may produce different types of results depending on the instruction type. These include the ALU result for arithmetic instructions, data read from memory for load instructions, or the PC + 1 value for CALL instructions. A dedicated multiplexer controlled by the WB_Sel_MEM signal selects the appropriate value at the MEM stage. This selected value is then provided to the

forwarding network and routed to the forwarding multiplexers connected to the Rs, Rt, and Rp paths.

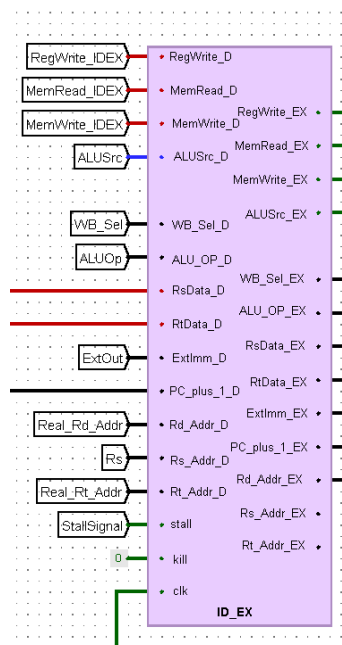
D. Design Impact

The use of these Forwarding Units allows the processor to resolve nearly all RAW hazards without stalling the pipeline. The only exception is the load-use hazard, in which a load instruction is immediately followed by an instruction that depends on its result. This specific case is handled by the Stall Unit. Overall, the forwarding mechanism significantly reduces pipeline stalls, improves the CPI (Cycles Per Instruction), and increases the overall throughput of the processor.

• ID-EX Pipeline Buffer

Finally, all decoded operands, extended immediates, destination register addresses, and gated control signals are captured into the ID_EX pipeline register on the rising clock edge.

The ID/EX pipeline register serves as the boundary between the Instruction Decode (ID) stage and the Execute (EX) stage. Its main function is to save all data and control signals generated during the Decode stage and deliver them to the Execute stage in a synchronized way on the rising edge of the clock.



This buffer stores the decoded control signals, including RegWrite, MemRead, MemWrite, ALUSrc, ALU_OP, and WB_Sel, along with the operand values read from the Register File (RsData and RtData), the extended immediate value, and the destination and source register addresses. It also carries the PC + 1 value required for CALL instructions and forwarding support.

To support correct pipelined execution, the ID/EX register responds to both stall and kill conditions. When a stall is asserted, the control signals stored in the ID/EX buffer are cleared, effectively inserting a pipeline bubble while preserving pipeline. This prevents the Execute stage from performing any operation during the stall cycle. When a kill signal is asserted, the instruction is similarly blocked to avoid incorrect execution after control hazards.

1.4.3. Execution Stage (EX)

The Execution (EX) stage is responsible for performing all arithmetic and logical operations required by the instruction. It is also the stage where effective addresses for memory operations are calculated. The central component of this stage is the Arithmetic Logic Unit (ALU).

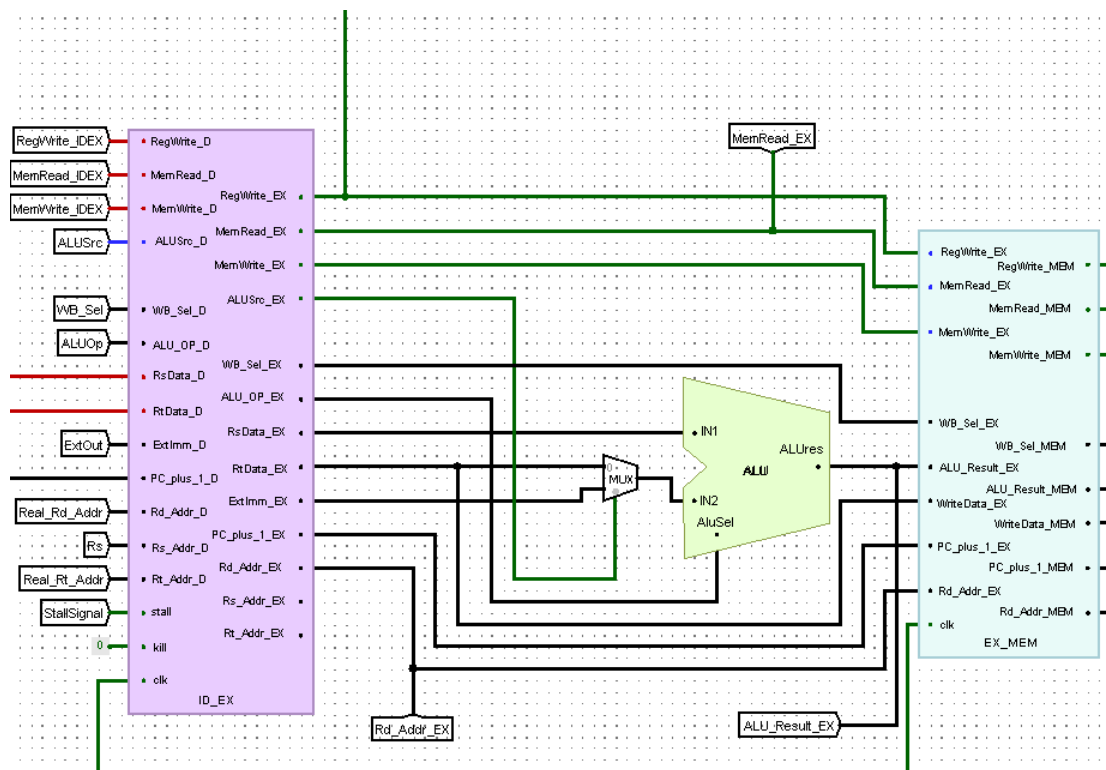


Figure 32: Execution Stage (EX)

The first ALU operand is obtained from the forwarded value of the Rs register, ensuring that the most recent data is used even if it has not yet been written back to the Register File. The second

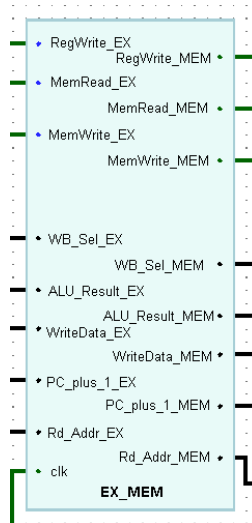
ALU operand is selected by a multiplexer controlled by the `ALUSrc_EX` signal. This multiplexer chooses between the forwarded `Rt` value and the 32-bit extended immediate value (`ExtImm_EX`). This design allows the same ALU hardware to support both register-to-register operations, such as `ADD` and `SUB`, and immediate-based instructions, such as `ADDI`, as well as address calculations for load and store instructions.

The operation performed by the ALU is selected using the 3-bit `ALU_OP_EX` control signal. This signal follows a standardized encoding scheme designed to minimize logic delay, where 000 corresponds to addition, 001 to subtraction, 010 to bitwise OR, 011 to bitwise NOR, and 100 to bitwise AND. Based on the value of this control signal, the ALU executes the required arithmetic or logical operation. Reusing the same addition opcode (000) for both arithmetic instructions and effective memory address calculations in load and store instructions reduces the complexity of the Control Unit. This design choice simplifies the control logic and contributes to a more efficient hardware implementation.

Once the computation is complete, the ALU result is distributed to multiple destinations. It is first latched into the EX/MEM pipeline register to be used in the following Memory stage. At the same time, the ALU result is forwarded to the forwarding network, allowing subsequent dependent instructions to use the value immediately without waiting for the write-back stage. This early availability of results is a key requirement for efficient pipelined execution.

- **EX-MEM Pipeline Buffer:**

The EX/MEM pipeline register buffers all outputs of the Execution stage, including the ALU result, the data to be written to memory for store instructions, the `PC + 1` value for `CALL` instructions, and the associated control signals. In the Memory (MEM) stage, the ALU result is used as the memory address for load and store operations, while the Data Memory produces `MemData_MEM` for load instructions. By isolating memory access to the MEM stage, the processor keeps the Execution stage fast and allows arithmetic operations to proceed without unnecessary delays.



A key design decision was made to strictly enforce predicated execution without compromising processor performance. In this architecture, the predicate register condition is evaluated early in the Instruction Decode (ID) stage. The RegWrite and MemWrite control signals are gated by the ExecuteEn signal during decoding, ensuring that instructions with a false predicate do not modify architectural state. As a result, the Execute stage remains transparent to the predication mechanism, and the ALU performs its computations independently of the predicate outcome. When the predicate evaluates to zero, the computed results are simply discarded and are not written to either memory or registers. This approach eliminates the need for additional gating logic within the ALU's critical path, thereby preserving the processor's maximum clock frequency while fully satisfying the conditional execution requirements defined by the instruction set architecture.

1.4.4. Memory Access Stage (MEM):

The Memory Access (MEM) stage interfaces with the processor's data storage and is responsible for handling load and store instructions. All memory-related operations are isolated in this stage to keep the Execution stage fast and to maintain clear pipeline separation.

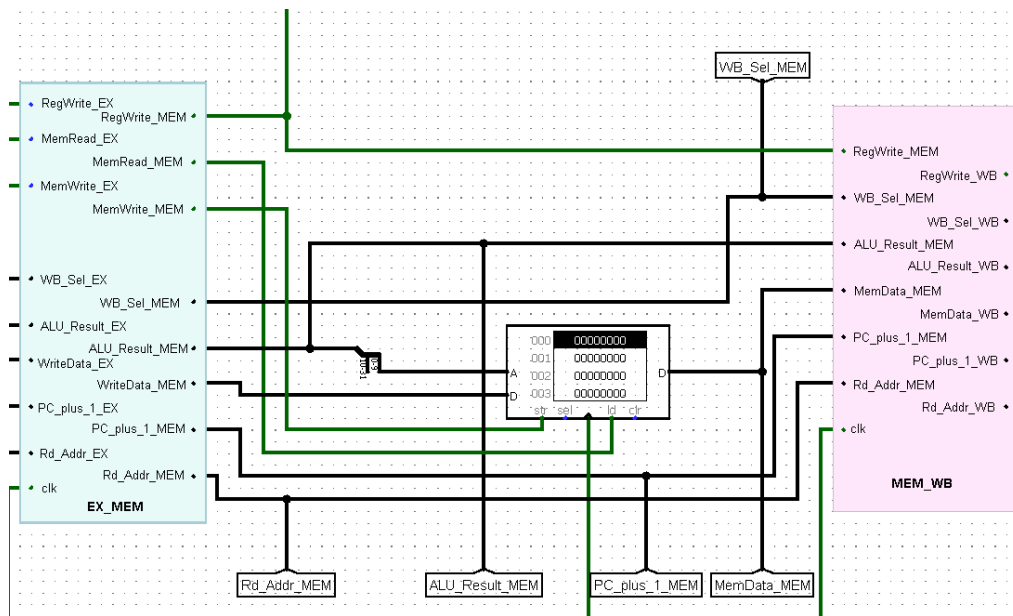


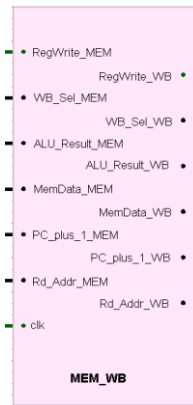
Figure 33: Memory Access Stage (MEM)

Consistent with our "Early Resolution" design, if the instruction's predicate register was zero, the MemWrite and MemRead signals arrive at this stage already gated to zero. This ensures that memory state is never modified by an instruction whose predicate is false.

A key design choice in this processor is the use of word addressable memory. To address the 1024 words in the 4 KB memory, only the 10 least significant bits of the ALU result are used as the memory address.

• MEM-WB Pipeline Buffer

The EX/MEM pipeline register plays a critical role in this stage by buffering the ALU result, which serves as the memory address, along with the data to be written for store instructions and the associated control signals. After the memory operation is completed, all relevant results—including the ALU result, memory read data, and the PC + 1 value for CALL instructions—are propagated to the MEM/WB pipeline register. This ensures proper synchronization and prepares the data for the final Write-Back stage.



After the memory access is completed, all relevant results—including the ALU result, the data read from memory, the PC + 1 value used for CALL instructions, and the destination register address (Rd)—are latched into the MEM/WB pipeline register. This register synchronizes the data with the system clock before it is forwarded to the Write-Back stage. It effectively holds the three possible sources for register write-back—ALU result, memory data, or link address—allowing the final selection to be performed in the last stage of the pipeline.

1.4.5. Write Back Stage (WB):

The Write-Back (WB) stage is the fifth and final stage of the pipeline. Its primary role is to select the correct result produced by earlier stages and commit it to the Register File, thereby permanently updating the architectural state of the processor.

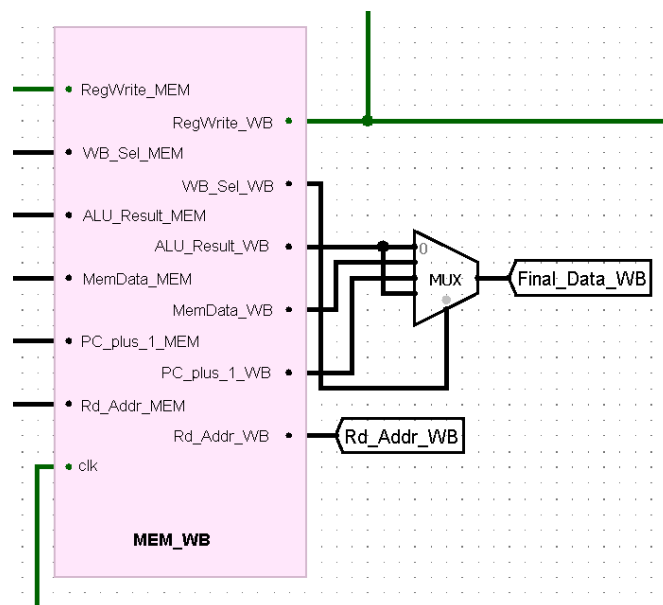


Figure 34: Write Back Stage (WB)

The central component of this stage is the write-back selection multiplexer. Since different instruction types generate different results, this 3-to-1 multiplexer is controlled by the WB_Sel_WB signal to choose the appropriate value to be written back. For arithmetic and logical instructions, the ALU result is selected. For load instructions, the data read from memory is selected. For CALL instructions, the PC + 1 value is selected, allowing the return address of the procedure to be stored in the Register File.

The output of this multiplexer, labeled Final_Data_WB, is routed back to the Instruction Decode stage through a global feedback path and connected to the DataIn port of the Register File. On the rising edge of the clock, the Register File writes this value into the destination register specified by Rd_Addr_WB, provided that the RegWrite_WB control signal is asserted. As established earlier, this signal depends on both the decoded opcode and the predicate evaluation performed in the Decode stage, ensuring that predicated-false instructions do not modify the processor state.

A key design decision in this processor is the commit-at-end strategy, where all architectural state updates—except for the Program Counter—occur exclusively in the Write-Back stage. This approach ensures that all instructions complete their execution uniformly, simplifying pipeline control and improving synchronization. Additionally, this structure provides a clean foundation for future support of exceptions or interrupts, as instructions only affect permanent state at the final stage.

While general-purpose registers (R1–R29 and R31) are updated during the Write-Back stage, the Program Counter (R30) is treated as a special case and is updated every cycle through the Fetch and Decode loop to maintain continuous instruction flow. The CALL instruction uniquely leverages the Write-Back stage to update R31 with the return address, fulfilling the procedure call requirements of the instruction set architecture.

2. Control Path & Control Signals

The control path is responsible for interpreting the 5-bit Opcode and generating the signals that coordinate the movement of data through the 5-stage pipeline. The control logic is decentralized: the main Control Unit resides in the Decode (ID) stage, while hazard-related logic (Stall and Kill units) and predicated-execution gating are distributed across the early

pipeline stages to preserve timing integrity and ensure correct pipelined execution. Control signals are generated combinatorially for the instruction in the Decode stage and then carried forward through the pipeline registers (ID/EX, EX/MEM, MEM/WB) so that each instruction retains its correct control context as it progresses through the pipeline.

1.1 Control Signals Description

The following signals are generated by the Control Unit to manage the datapath:

- **RegWrite (1 bit):**
Enables writing the final result into the Register File during the Write-Back stage.
- **ALUSrc (1 bit):**
Selects the source of the second ALU operand
(0: register data, 1: extended immediate).
- **RegDst (1 bit):**
Selects the destination register index
(0: Rd field, 1: hardwired R31 for CALL instructions).
- **MemRead (1 bit):**
Enables reading data from the Data Memory (used by LW instructions).
- **MemWrite (1 bit):**
Enables writing data to the Data Memory (used by SW instructions).
- **RegReadSrc2 (1 bit):**
Selects the source of the second register read address
(0: Rt, 1: Rd for Store instructions).
- **WB_Sel (2 bits):**
Selects the data source for the Write-Back stage: 00: ALU result, 01: Memory read data, 10: PC + 1 (CALL instruction)
- **PCSrc (2 bits):**
Determines the next Program Counter value: 00: PC + 1, 01: Jump/Call target address, 10: Register value (JR)
- **ExtSel (2 bits):**
Controls immediate extension mode: 00: Zero-extension (12-bit), 01: Sign-extension (12-bit), 10: Sign-extension (22-bit)

- **ALU_OP (3 bits):**

Specifies the arithmetic or logical operation performed by the ALU (ADD, SUB, OR, NOR, AND).

- **ExecuteEn (1 bit):**

Generated by the Predicate Unit; enables or disables instruction execution based on the predicate register.

- **StallSignal (1 bit):**

Generated by the Stall Unit; freezes pipeline stages to handle load-use hazards.

- **killSignal (1 bit):**

Generated by the Kill Unit; flushes incorrectly fetched instructions following control hazards.

- **fwd_rs (2 bits):**

Generated by the Forwarding Unit to control the selection of the source for the Rs operand. It selects the most recent valid data from the Register File, EX stage, MEM stage, or WB stage to resolve read-after-write (RAW) data hazards.

- **fwd_rt (2 bits):**

Generated by the Forwarding Unit to control the selection of the source for the Rt operand. Similar to fwd_rs, it prioritizes forwarded results from later pipeline stages to ensure correct operand values.

- **fwd_rp (2 bits):**

Generated by the Forwarding Unit to control the selection of the source for the predicate register operand Rp. This signal ensures correct predicate evaluation when the predicate value is produced by an instruction still in the pipeline.

- **WB_Sel_MEM (2 bits):**

A feedback control signal used during the Decode stage to select the appropriate value for forwarding from the Memory stage. Depending on its value, the forwarded data may be the ALU result, memory read data, or the link address (PC + 1) generated by a CALL instruction

.

2.2. Control Signals Truth Table

The truth table shown above represents the hardwired combinational logic implemented in the ControlUnit.v module. For each instruction opcode, the Control Unit generates a fixed set of control signals that determine how the instruction is processed through the pipeline. These signals control register access, ALU operation, memory interaction, write-back behavior, and program counter updates.

Instruction	Opcode	ALUSrc	ALU_OP	PCSrc	ExtSel	RegW
ADD	0	0	000	00	XX	1
SUB	1	0	001	00	XX	1
OR	2	0	010	00	XX	1
NOR	3	0	011	00	XX	1
AND	4	0	100	00	XX	1
ADDI	5	1	000	00	01	1
ORI	6	1	010	00	00	1
NORI	7	1	011	00	00	1
ANDI	8	1	100	00	00	1
LW	9	1	000	00	01	1
SW	10	1	000	00	01	0
J	11	X	XXX	01	10	0
CALL	12	X	XXX	01	10	1
JR	13	0	XXX	10	XX	0

Instruction	RegDst	MemR	MemW	WBSEL	RegReadSrc2
ADD	0	0	0	00	0
SUB	0	0	0	00	0
OR	0	0	0	00	0
NOR	0	0	0	00	0
AND	0	0	0	00	0
ADDI	0	0	0	00	0
ORI	0	0	0	00	0
NORI	0	0	0	00	0

ANDI	0	0	0	00	0
LW	0	1	0	01	0
SW	0	0	1	XX	1
J	0	0	0	XX	0
CALL	1	0	0	10	0
JR	0	0	0	XX	0

2.3. Boolean Equations

The processor uses Boolean equations only for non-trivial control logic that cannot be expressed solely through opcode decoding. These equations implement predicated execution, hazard **detection**, control-flow flushing, and final permission gating. All other control signals are generated directly by the Control Unit using the opcode truth table.

Predication Logic

$$\text{ExecuteEn} = (\text{Rp_addr} == 0) \text{ OR } (\text{Rp_data} \neq 0)$$

This equation determines whether an instruction is allowed to execute. If the predicate register is R0, execution is unconditional. Otherwise, execution is enabled only when the value stored in the predicate register is non-zero.

Stall Logic (Load-Use Hazard Detection)

$$\text{Stall} = \text{MemRead_EX} \text{ AND } ((\text{Rd_EX} == \text{Rs_ID}) \text{ OR } (\text{Rd_EX} == \text{Rt_ID}) \text{ OR } (\text{Rd_EX} == \text{Rp_ID}))$$

This equation detects load-use data hazards. A stall is asserted when an instruction in the Execute stage performs a memory read and the following instruction in the Decode stage depends on the same destination register as a source or predicate register.

Kill Logic

$$\text{Kill} = (\text{PCSrc_ID} \neq 00) \cdot \text{ExecuteEn}$$

This equation generates the flush signal for control hazards. When a jump, call, or register jump instruction is selected and the predicate condition is satisfied, the instruction fetched in the Fetch stage is invalid and must be replaced with a NOP.

Actual Permission (Control Gating) Logic

$$\text{RegWrite_Actual} = \text{RegWrite_CU} \cdot \text{ExecuteEn} \cdot \sim(\text{Stall})$$

This equation defines the final register write-back permission. Even if the Control Unit asserts RegWrite, the write operation occurs only when the predicate condition is true and no stall is active. Similar gating logic is applied to memory read and write signals to prevent unintended state updates.

Memory Write Permission Logic

$$\text{MemWrite_Actual} = \text{MemWrite_CU} \cdot \text{ExecuteEn} \cdot \sim(\text{Stall})$$

This equation defines the final permission for memory write operations. Even if the Control Unit asserts the MemWrite signal, the data memory is modified only if the predicate condition is satisfied and no pipeline stall is active. When the predicate evaluates to false or a stall is asserted, the memory write-enable signal is forced to zero, preventing unintended updates to memory state. This ensures correct handling of Store Word (SW) instructions under predication and hazard conditions.

Forwarding Unit Selection Logic

The forwarding network resolves Read-After-Write (RAW) data hazards by selecting the most recent available value for each source operand. The control signals for the forwarding multiplexers (fwd_rs, fwd_rt, and fwd_rp) are generated using priority-based Boolean logic. The logic below illustrates the selection for the Rs source operand; identical logic is applied to the Rt and predicate (Rp) paths.

Forward from Execute Stage (Highest Priority)

$$\text{If } (\text{RegWrite_EX} \text{ AND } (\text{Rd_EX} == \text{Rs_ID}) \text{ AND } (\text{Rs_ID} \neq 0)) \rightarrow \text{fwd_rs} = 01$$

Forward from Memory Stage

Else if (RegWrite_MEM AND (Rd_MEM == Rs_ID) AND (Rs_ID ≠ 0)) → fwd_rs = 10

Forward from Write-Back Stage

Else if (RegWrite_WB AND (Rd_WB == Rs_ID) AND (Rs_ID ≠ 0)) → fwd_rs = 11

No Forwarding Required

Else → fwd_rs = 00

This guarantees that the most recently produced value is always selected, with precedence given to the Execute stage, followed by the Memory stage, and finally the Write-Back stage. By extending this logic to the predicate register path (Rp), the processor ensures correct predicate evaluation and control-flow decisions without introducing unnecessary stalls.

- **Note :** Because this processor is implemented as a **5-stage pipeline**, control is distributed across stages via pipeline registers rather than a centralized finite state machine. Therefore, the "state" of the processor is represented by the collection of instructions and their associated control signals currently residing in the five pipeline buffers.

4. Simulation & Testing

This section validates the correctness of the designed pipelined processor through functional simulation. A set of carefully chosen test programs was executed to verify normal instruction execution, data hazard handling, control hazard resolution, and predicated execution behavior. Simulation was performed using the HDL simulator with waveform observation to confirm both data values and control signals across pipeline stages.

- **Correctness of Individual Components**
- **ALU Testbench Simulation**

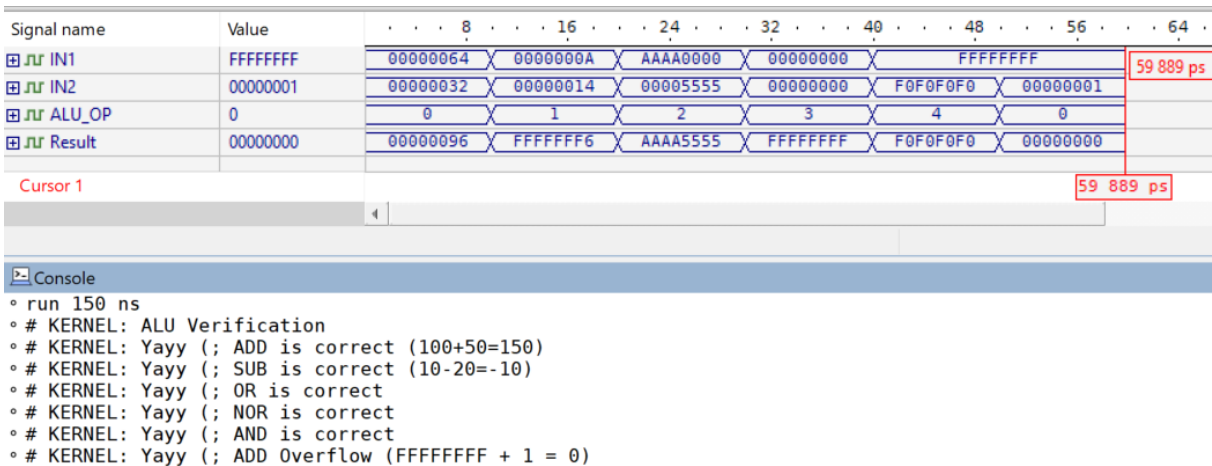


Figure 35: ALU TB Simulation

This simulation validates the correctness of the Arithmetic Logic Unit (ALU) by applying a sequence of predefined input vectors and checking that the produced output matches the expected result for each operation. In the waveform window, IN1 and IN2 represent the two 32-bit ALU operands, ALU_OP represents the operation selector, and Result is the 32-bit ALU output. As the simulation progresses, the testbench changes IN1, IN2, and ALU_OP to cover all supported functions, and the output waveform shows the corresponding Result values for each case.

The console output confirms that each test case passed successfully. The testbench first verifies addition by applying operands 100 and 50 and checking that the result equals 150. It then verifies subtraction using operands 10 and 20, expecting a result of -10 in 32-bit two's complement representation. Logical operations are also tested: OR, NOR, and AND are executed using operand

patterns that make the expected bitwise results easy to verify. Finally, the testbench checks an overflow wrap-around case for unsigned addition by applying $IN1 = 0xFFFFFFFF$ and $IN2 = 1$; the correct 32-bit result is $0x00000000$ due to natural wrap-around in fixed-width arithmetic. The message “ADD Overflow ($FFFFFFFF + 1 = 0$)” indicates that this wrap-around behavior was correctly observed.

- **Data Memory Testbench Simulation**

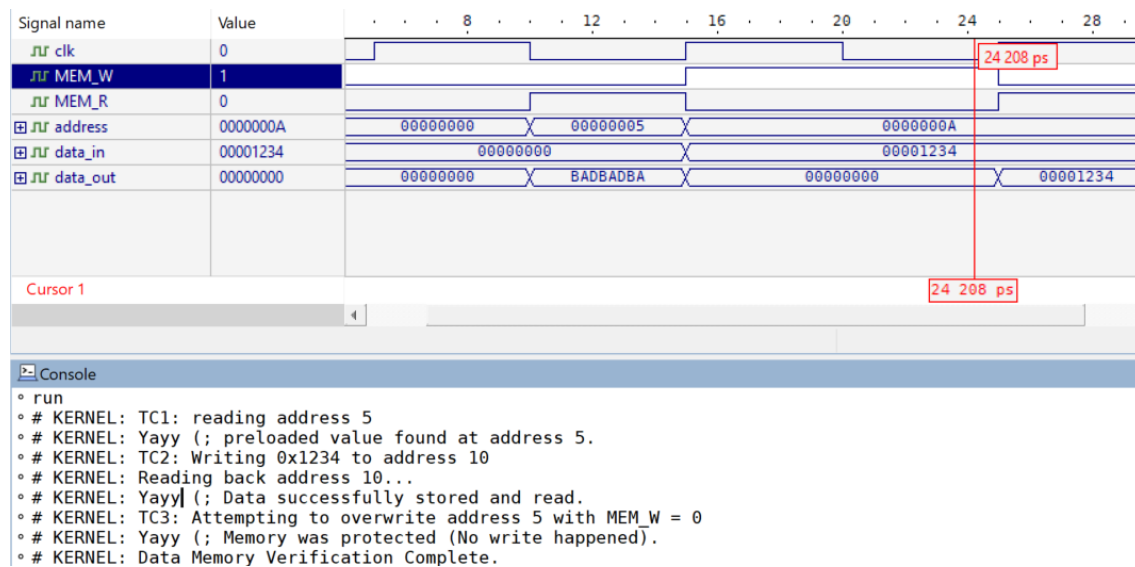


Figure 36: Data Memory TB

In the first test case (TC1), a read operation is performed at address 5 with MEM_R asserted and MEM_W deasserted. The waveform shows that data_out correctly reflects the preloaded value stored at that address. This confirms that the Data Memory supports correct read functionality and that initial memory contents are accessible.

In the second test case (TC2), a write operation is tested by asserting MEM_W and placing the value 0x1234 on the data_in bus while the address is set to 10. The write occurs on the rising edge of the clock. A subsequent read from address 10 shows that data_out returns the same value, verifying correct synchronous write behavior and successful data storage.

The third test case (TC3) validates memory protection. An attempt is made to write to address 5 while MEM_W is deasserted. As expected, no memory update occurs, and the original value at

that address remains unchanged. This behavior is confirmed by both the waveform and the console output, ensuring that unintended writes are properly blocked.

- **Extender Testbench Simulation**

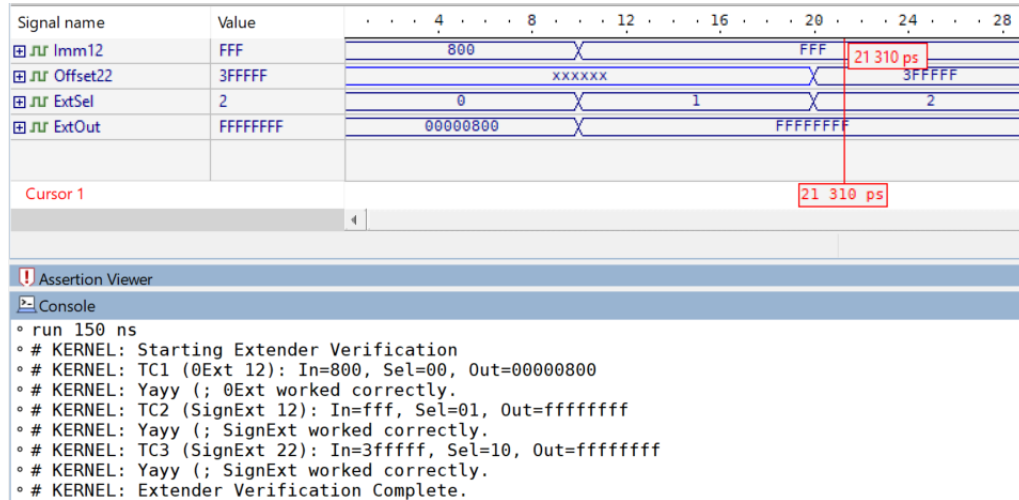


Figure 37: Extender TB

The testbench applies three main test cases that correspond to the supported extension modes. In TC1, ExtSel is set to 00 to perform zero-extension of a 12-bit immediate. The input Imm12 is set to 0x800, and the output becomes 0x00000800, confirming that the module correctly pads the upper bits with zeros. In TC2, ExtSel is set to 01 to perform sign-extension of a 12-bit immediate. The input Imm12 is set to 0xFFF (which represents -1 in 12-bit two's complement), and the output becomes 0xFFFFFFFF, confirming correct sign propagation into the upper bits. In TC3, ExtSel is set to 10 to perform sign-extension of a 22-bit offset. The input Offset22 is set to 0x3FFFFFF (which represents -1 in 22-bit two's complement), and the output again becomes 0xFFFFFFFF, confirming that sign-extension is correctly applied for jump/branch offsets.

The console output reports that all test cases passed successfully and matches the waveform transitions. Overall, the results confirm that the Extender module produces the correct 32-bit extended values for zero-extended immediates, sign-extended immediates, and sign-extended offsets.

- **Predicate Unit Testbench Simulation**

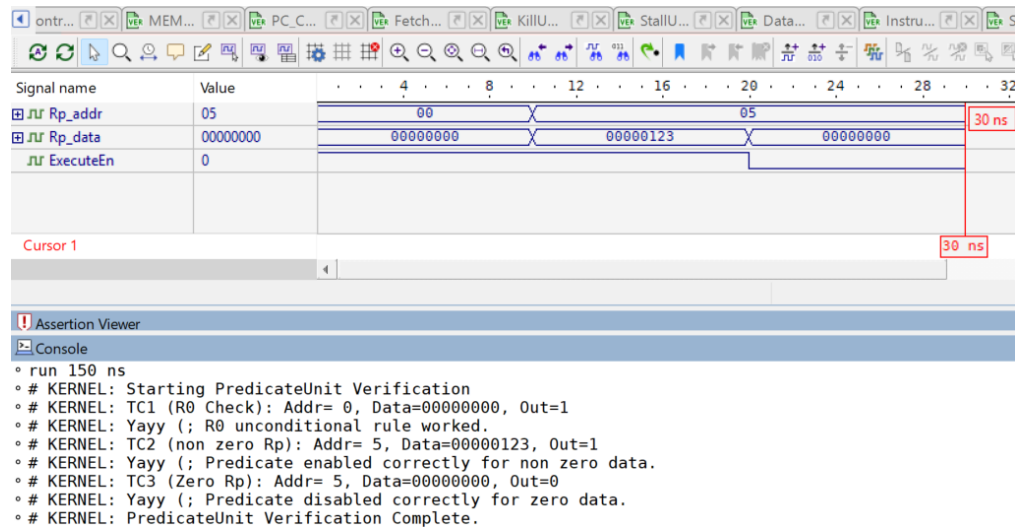


Figure 38: Predicate Unit TB

In the first test case (TC1), `Rp_addr` is set to 0. By ISA definition, register R0 represents an unconditional predicate. As shown in the waveform and confirmed by the console output, `ExecuteEn` is asserted to 1 regardless of the value of `Rp_data`. This verifies the rule that instructions with `Rp = R0` always execute.

In the second test case (TC2), `Rp_addr` is set to a non-zero register (R5), and `Rp_data` is set to a non-zero value (0x00000123). The Predicate Unit correctly asserts `ExecuteEn` to 1, indicating that the predicate condition is satisfied and the instruction is allowed to proceed. This confirms correct handling of enabled predicated execution.

In the third test case (TC3), `Rp_addr` remains R5, but `Rp_data` is set to 0x00000000. In this case, `ExecuteEn` is deasserted to 0, correctly disabling the instruction. This demonstrates that when the predicate register is non-zero but its value is zero, the instruction is treated as inactive and will be converted into a NOP through control gating.

The console output confirms that all three test cases passed successfully. Overall, the simulation verifies that the Predicate Unit correctly implements the predication rule:

$$\text{ExecuteEn} = (\text{Rp_addr} == 0) \text{ OR } (\text{Rp_data} \neq 0).$$

- **Forwarding Unit Testbench Simulation**

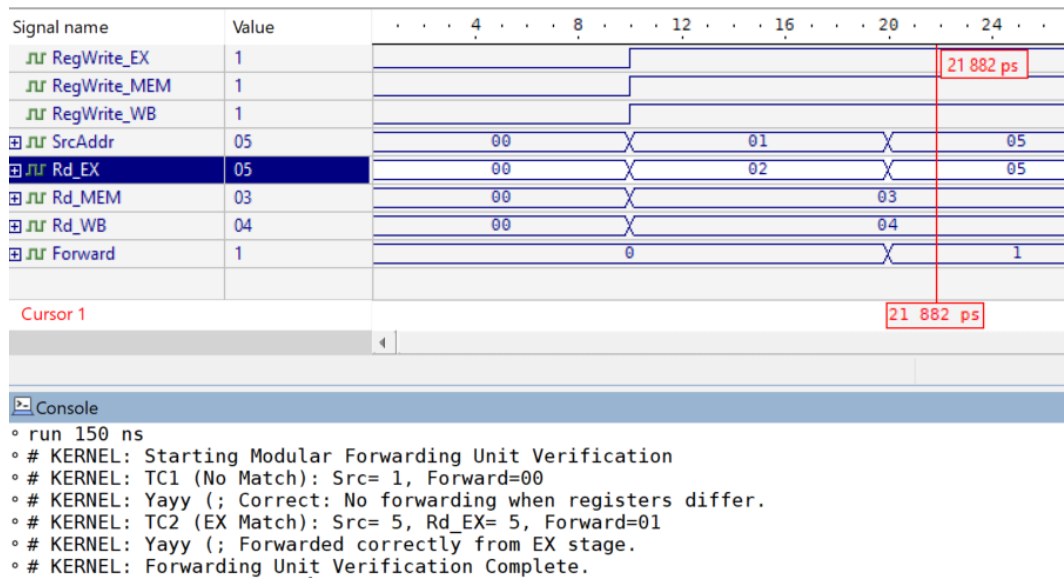


Figure 39: Forwarding Unit TB

In the first test case (TC1), the source register (SrcAddr = 1) does not match any of the destination registers in the EX, MEM, or WB stages. As expected, the Forward signal remains 00, indicating that no forwarding is required and the operand should be taken directly from the Register File. This confirms correct behavior when no data hazard is present.

In the second test case (TC2), the source register is set to 5, which matches the destination register in the Execute stage (Rd_EX = 5), with RegWrite_EX asserted. The Forward signal changes to 01, selecting the ALU result from the EX stage. This demonstrates that the Forwarding Unit correctly detects an EX-stage RAW hazard and gives it the highest priority, ensuring that the most recent data is used.

The console output confirms that both test cases passed successfully.

- Comprehensive Test program in Assembly language and binary format:

- ❖ **Assembly & Machine Code :**

2802000A → ADDI R1, R0, 10

28041005 → ADDI R2, R1, 5

4006200F → ANDI R3, R2, 15

300800FF → ORI R4, R0, 255

380A4000 → NORI R5, R4, 0

080C4100 → SUB R6, R4, R2

200E6080 → AND R7, R6, R1

10101100 → OR R8, R1, R2

18121000 → NOR R9, R1, R0

500E0014 → SW R7, 20(R0)

48140014 → LW R10, 20(R0)

0016A000 → ADD R11, R10, R0 (load-use stall expected)

28180000 → ADDI R12, R0, 0

281A0001 → ADDI R13, R0, 1

031C1100 → ADD (predicated by R12) → SKIP

035E1100 → ADD (predicated by R13) → EXECUTE

59800004 → J +4 (predicated by R12) → SKIP (no jump)

58000003 → J +3 (predicated by R0) → TAKEN to PC 21

28280063 → ADDI R20, R0, 99 (must be killed)

282A0063 → ADDI R21, R0, 99

282C0063 → ADDI R22, R0, 99 (must be killed)

60000001 → CALL +1 (predicated by R0) → TAKEN to PC 23, R31 = 22

58000001 → J +2 (predicated by R0) → executed after JR returns

00300000 → ADD R24, R0, R0

6801F000 → JR R31, R0 → return to PC 22

0000DEAD → STOP

❖ **The expected Results:**

Expected Results With Commit Cycle (Cycle count starts at 1)

R1 = 10 → at cycle 5

R2 = 15 → at cycle 6

R3 = 15 → at cycle 7

R4 = 255 → at cycle 8

R5 = 0xFFFFFFFF00 → at cycle 9

R6 = 240 → at cycle 10

R7 = 0 → at cycle 11

R8 = 15 → at cycle 12

R9 = 0xFFFFFFFF5 → at cycle 13

R10 = 0 → at cycle 15

Reason: LW is fetched after the SW and its WB happens later.

R11 = 0 → at cycle 17

Reason: load-use stall between LW (R10) and ADD (R11), so ADD's WB is delayed by 1 cycle.

R12 = 0 → at cycle 18

R13 = 1 → at cycle 19

R14 unchanged (predicate R12=0) → no commit

Reason: ExecuteEn = 0.

R15 = 25 → at cycle 21

Reason: predicated instruction with Rp=R13 executes normally.

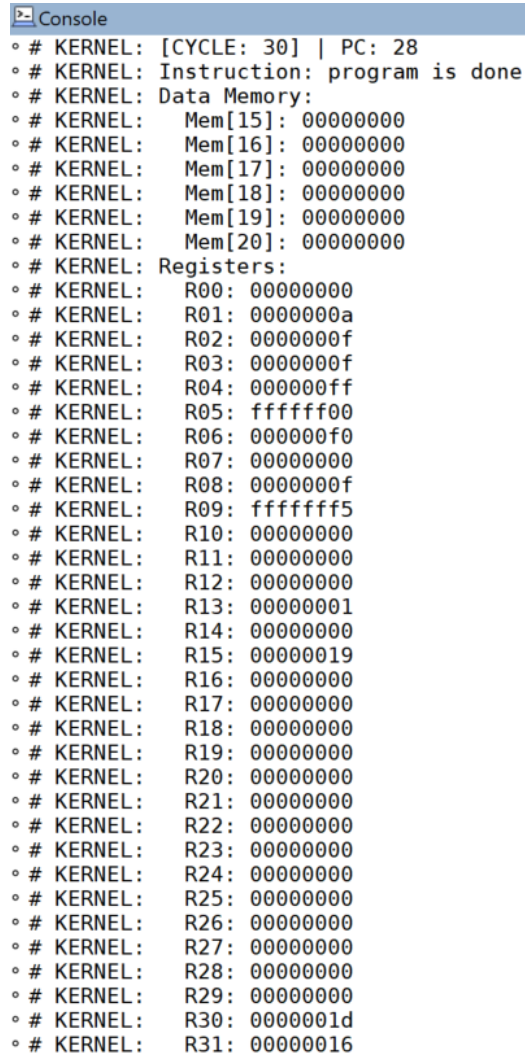
R31 = 22 → at cycle 24

Reason: CALL saves the return address (PC + 1) into R31 during Write-Back after jump resolution.

R24 = 0 → at cycle 29

Reason: this instruction is inside the CALL “function body”, so it commits after the jump/call/jr control-flow.

❖ Actual Final Results:



```
Console
° # KERNEL: [CYCLE: 30] | PC: 28
° # KERNEL: Instruction: program is done
° # KERNEL: Data Memory:
° # KERNEL:   Mem[15]: 00000000
° # KERNEL:   Mem[16]: 00000000
° # KERNEL:   Mem[17]: 00000000
° # KERNEL:   Mem[18]: 00000000
° # KERNEL:   Mem[19]: 00000000
° # KERNEL:   Mem[20]: 00000000
° # KERNEL: Registers:
° # KERNEL:   R00: 00000000
° # KERNEL:   R01: 0000000a
° # KERNEL:   R02: 0000000f
° # KERNEL:   R03: 0000000f
° # KERNEL:   R04: 000000ff
° # KERNEL:   R05: ffffffff00
° # KERNEL:   R06: 000000f0
° # KERNEL:   R07: 00000000
° # KERNEL:   R08: 0000000f
° # KERNEL:   R09: ffffffff5
° # KERNEL:   R10: 00000000
° # KERNEL:   R11: 00000000
° # KERNEL:   R12: 00000000
° # KERNEL:   R13: 00000001
° # KERNEL:   R14: 00000000
° # KERNEL:   R15: 00000019
° # KERNEL:   R16: 00000000
° # KERNEL:   R17: 00000000
° # KERNEL:   R18: 00000000
° # KERNEL:   R19: 00000000
° # KERNEL:   R20: 00000000
° # KERNEL:   R21: 00000000
° # KERNEL:   R22: 00000000
° # KERNEL:   R23: 00000000
° # KERNEL:   R24: 00000000
° # KERNEL:   R25: 00000000
° # KERNEL:   R26: 00000000
° # KERNEL:   R27: 00000000
° # KERNEL:   R28: 00000000
° # KERNEL:   R29: 00000000
° # KERNEL:   R30: 0000001d
° # KERNEL:   R31: 00000016
```

Figure 40: Testing - Actual Final Results

In a 5-stage pipelined processor, the total number of execution cycles is generally given by

Total Cycles = $N + 4 + \text{stall cycles} + \text{flush cycles}$,

where N represents the number of dynamic instructions that actually enter the pipeline.

In this program, several control-flow instructions (J, CALL, and JR) are executed. Each taken jump causes the instruction that was already fetched to be flushed and replaced by a NOP, introducing a one-cycle control hazard penalty. As a result, although the Program Counter continues to advance, not all sequential instruction addresses correspond to useful instruction execution. The simulation terminates at **cycle 30**, which correctly reflects the total execution time including pipeline fill, stalls, and control-hazard flushes. Therefore, the total number of cycles is **30**, not 31, because jump instructions alter the normal sequential flow and eliminate some

instructions before they can commit architectural state

- **Final Pipelied Processor Waveform :**

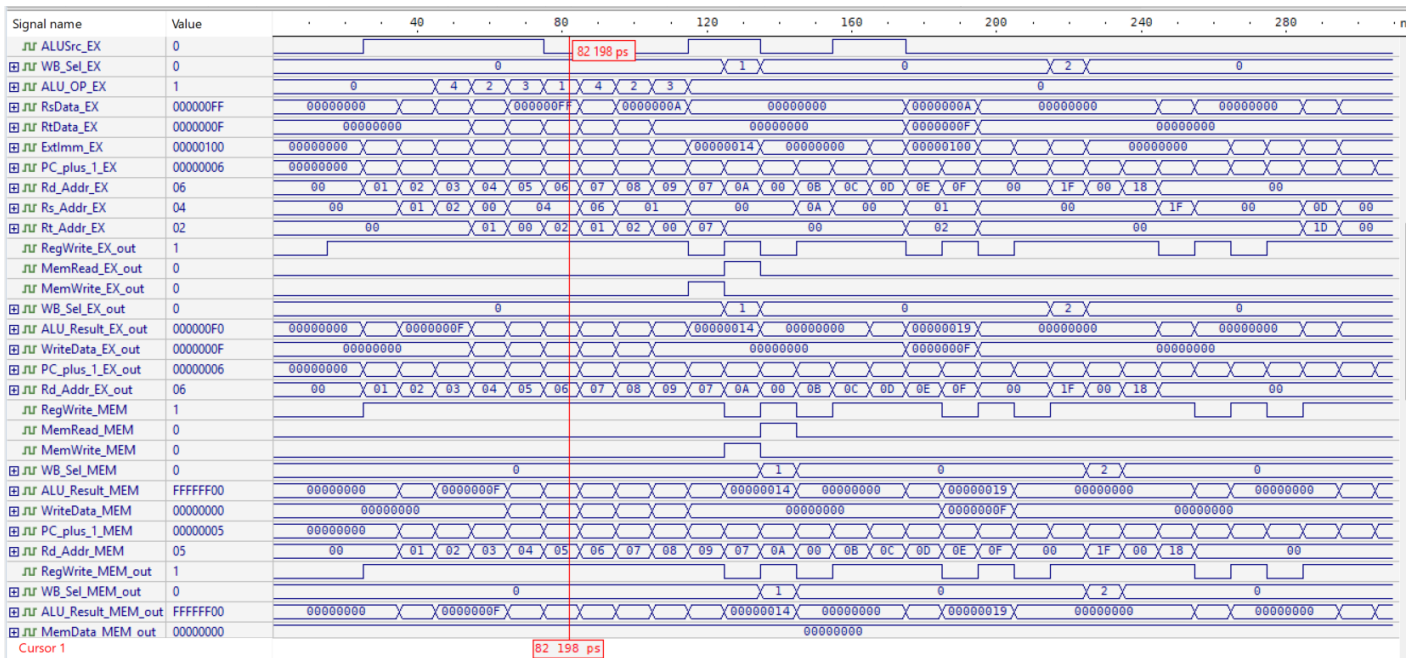
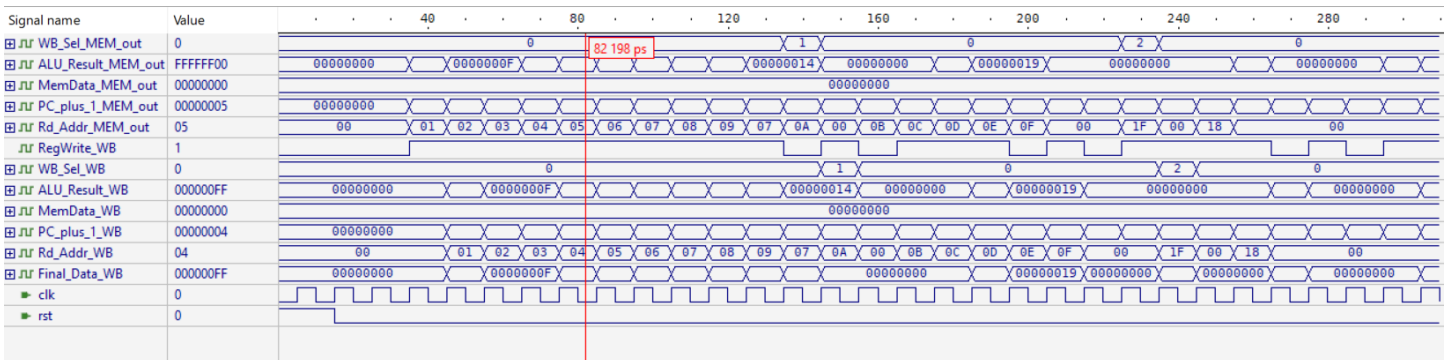
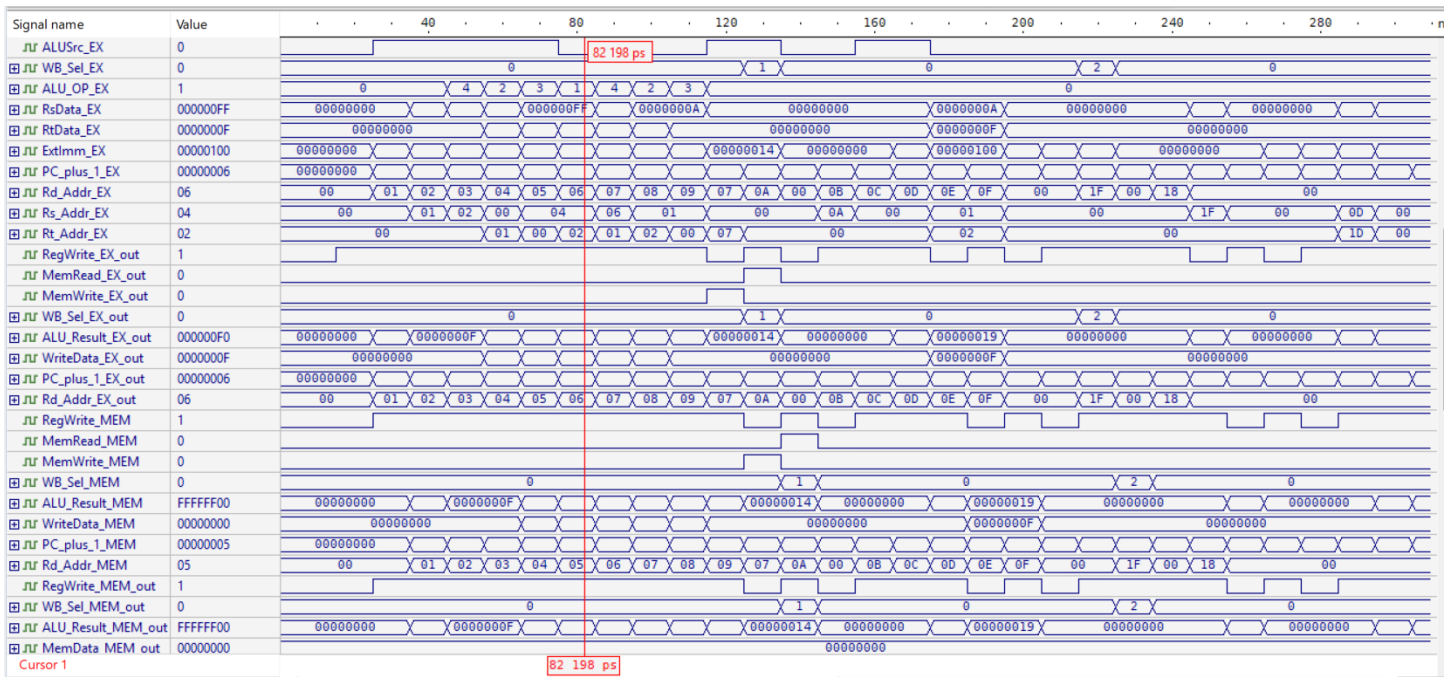


Figure 41: pipelined Processor Waveform



The final waveform illustrates the complete operation of the 5-stage pipelined processor while executing the full test program. The signals shown span all pipeline stages (IF, ID, EX, MEM, WB) and demonstrate correct instruction flow, hazard handling, and control-flow resolution over time.

At the top of the waveform, the **instruction fetch and decode signals** (Instruction_F, Instruction_D, current_pc_D, PC_plus_1_D) show continuous instruction issue during normal execution. The Program Counter increments sequentially until a control instruction is detected. When a jump or call instruction reaches the Decode stage, the **PCSrc_D** signal changes and the **kill** signal is asserted, causing the instruction already fetched on the wrong path to be replaced with a NOP. This confirms correct **early control-hazard resolution** with a one-cycle penalty.

The **stall signal** becomes active when a load-use dependency is detected. In this case, the Decode and Fetch stages are frozen for one cycle while a bubble is injected into the pipeline. This behavior is visible in the waveform as a temporary hold of IF/ID values and suppressed control signals entering the ID/EX register. Once the stall is released, execution resumes normally, confirming correct load-use hazard handling.

The **Execute stage signals** (ALUSrc_EX, ALU_OP_EX, RsData_EX, RtData_EX, ExtImm_EX) show correct operand selection and ALU operation decoding. Forwarded values appear on RsData_EX and RtData_EX when dependencies exist, demonstrating that results from the EX, MEM, or WB stages are correctly bypassed back to the ALU without waiting for register write-back.

In the **MEM stage**, the ALU result is used as the memory address, while MemWrite_MEM and MemRead_MEM control the data memory. The waveform shows memory writes occurring only when MemWrite is asserted and gated by ExecuteEn, preventing unintended updates. Memory reads appear one stage later and are correctly forwarded or written back.

The **Write-Back stage** signals (WB_Sel_WB, Final_Data_WB, Rd_Addr_WB, RegWrite_WB) confirm that the correct result source is selected for each instruction. Arithmetic instructions write the ALU result, load instructions write memory data, and CALL instructions write PC+1 into R31. The waveform shows register updates occurring only on valid instructions, while predicated-false and killed instructions produce no register writes.

Finally, the global clock and reset signals confirm fully synchronous operation. All pipeline registers advance on the rising clock edge, and no spurious writes or memory accesses occur after the program terminates. The final architectural state observed in the waveform matches the expected register and memory values derived analytically.

Overall, the waveform validates correct pipelined execution, proper handling of data hazards through forwarding and stalling, precise control-hazard resolution using kill logic, and accurate implementation of predicated execution. This confirms that the processor operates as designed under realistic instruction sequences.

❖ Complete Program Run from Cycle 1 to Cycle 30:

Console

```
◦ # KERNEL:
◦ # KERNEL: [CYCLE: 1] | PC: 0
◦ # KERNEL: Instruction: 2802000a
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 00000000
◦ # KERNEL:   R02: 00000000
◦ # KERNEL:   R03: 00000000
◦ # KERNEL:   R04: 00000000
◦ # KERNEL:   R05: 00000000
◦ # KERNEL:   R06: 00000000
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 00000000
◦ # KERNEL:   R09: 00000000
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000001
```

 Console

```
◦ # KERNEL: [CYCLE: 2] | PC: 1
◦ # KERNEL: Instruction: 28041005
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 00000000
◦ # KERNEL:   R02: 00000000
◦ # KERNEL:   R03: 00000000
◦ # KERNEL:   R04: 00000000
◦ # KERNEL:   R05: 00000000
◦ # KERNEL:   R06: 00000000
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 00000000
◦ # KERNEL:   R09: 00000000
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000002
◦ # KERNEL:   R31: 00000000
```

```

Console
◦ # KERNEL: [CYCLE: 3] | PC: 2
◦ # KERNEL: Instruction: 4006200f
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 00000000
◦ # KERNEL:   R02: 00000000
◦ # KERNEL:   R03: 00000000
◦ # KERNEL:   R04: 00000000
◦ # KERNEL:   R05: 00000000
◦ # KERNEL:   R06: 00000000
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 00000000
◦ # KERNEL:   R09: 00000000
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000003
◦ # KERNEL:   R31: 00000000

```

```

> Console
◦ # KERNEL: [CYCLE: 4] | PC: 3
◦ # KERNEL: Instruction: 300800ff
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 00000000
◦ # KERNEL:   R02: 00000000
◦ # KERNEL:   R03: 00000000
◦ # KERNEL:   R04: 00000000
◦ # KERNEL:   R05: 00000000
◦ # KERNEL:   R06: 00000000
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 00000000
◦ # KERNEL:   R09: 00000000
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000004
◦ # KERNEL:   R31: 00000000

```

```

Console
◦ # KERNEL: [CYCLE: 5] | PC: 4
◦ # KERNEL: Instruction: 380a4000
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 00000000
◦ # KERNEL:   R03: 00000000
◦ # KERNEL:   R04: 00000000
◦ # KERNEL:   R05: 00000000
◦ # KERNEL:   R06: 00000000
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 00000000
◦ # KERNEL:   R09: 00000000
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000005
◦ # KERNEL:   R31: 00000000

```

```

>- Console
◦ # KERNEL: [CYCLE: 6] | PC: 5
◦ # KERNEL: Instruction: 080c4100
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 00000000
◦ # KERNEL:   R04: 00000000
◦ # KERNEL:   R05: 00000000
◦ # KERNEL:   R06: 00000000
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 00000000
◦ # KERNEL:   R09: 00000000
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000006
◦ # KERNEL:   R31: 00000000

```

 Console

```
◦ # KERNEL: [CYCLE: 7] | PC: 6
◦ # KERNEL: Instruction: 200e6080
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 00000000
◦ # KERNEL:   R05: 00000000
◦ # KERNEL:   R06: 00000000
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 00000000
◦ # KERNEL:   R09: 00000000
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000007
◦ # KERNEL:   R31: 00000000
```

```

Console
◦ # KERNEL: [CYCLE: 8] | PC: 7
◦ # KERNEL: Instruction: 10101100
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: 00000000
◦ # KERNEL:   R06: 00000000
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 00000000
◦ # KERNEL:   R09: 00000000
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000008
◦ # KERNEL:   R31: 00000000

```

```

Console
◦ # KERNEL: [CYCLE: 9] | PC: 8
◦ # KERNEL: Instruction: 18121000
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 00000000
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 00000000
◦ # KERNEL:   R09: 00000000
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000009
◦ # KERNEL:   R31: 00000000

```

```
Console
◦ # KERNEL: [CYCLE: 10] | PC: 9
◦ # KERNEL: Instruction: 500e0014
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 00000000
◦ # KERNEL:   R09: 00000000
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 0000000a
◦ # KERNEL:   R31: 00000000
```

```

Console
◦ # KERNEL: [CYCLE: 11] | PC: 10
◦ # KERNEL: Instruction: 48140014
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 00000000
◦ # KERNEL:   R09: 00000000
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 0000000b
◦ # KERNEL:   R31: 00000000

```

Console

```
◦ # KERNEL: [CYCLE: 12] | PC: 11
◦ # KERNEL: Instruction: 0016a000
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: 00000000
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 0000000c
◦ # KERNEL:   R31: 00000000
```

Console

```
◦ # KERNEL: [CYCLE: 13] | PC: 11
◦ # KERNEL: Instruction: 0016a000
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 0000000c
◦ # KERNEL:   R31: 00000000
```

 Console

```
◦ # KERNEL: [CYCLE: 14] | PC: 12
◦ # KERNEL: Instruction: 28180000
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 0000000d
◦ # KERNEL:   R31: 00000000
```

```
Console
◦ # KERNEL: [CYCLE: 15] | PC: 13
◦ # KERNEL: Instruction: 281a0001
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 0000000e
◦ # KERNEL:   R31: 00000000
```

 Console

```
◦ # KERNEL: [CYCLE: 16] | PC: 14
◦ # KERNEL: Instruction: 031c1100
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 0000000f
◦ # KERNEL:   R31: 00000000
```

```

Console
◦ # KERNEL: [CYCLE: 17] | PC: 15
◦ # KERNEL: Instruction: 035e1100
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000010
◦ # KERNEL:   R31: 00000000

```

Console

```
◦ # KERNEL: [CYCLE: 18] | PC: 16
◦ # KERNEL: Instruction: 59800004
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000000
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000011
◦ # KERNEL:   R31: 00000000
```

Console

```
◦ # KERNEL: [CYCLE: 19] | PC: 20
◦ # KERNEL: Instruction: 00000000
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000001
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000015
◦ # KERNEL:   R31: 00000000
```

Console

```
◦ # KERNEL: [CYCLE: 20] | PC: 21
◦ # KERNEL: Instruction: 60000001
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000001
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000000
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000016
◦ # KERNEL:   R31: 00000000
```

 Console

```
◦ # KERNEL: [CYCLE: 21] | PC: 22
◦ # KERNEL: Instruction: 00000000
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000001
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000019
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000017
◦ # KERNEL:   R31: 00000000
```

 Console

```
◦ # KERNEL: [CYCLE: 22] | PC: 23
◦ # KERNEL: Instruction: 00300000
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000001
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000019
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000018
◦ # KERNEL:   R31: 00000000
```

Console

```
◦ # KERNEL: [CYCLE: 23] | PC: 24
◦ # KERNEL: Instruction: 6801f000
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000001
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000019
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000019
◦ # KERNEL:   R31: 00000000
```

```

> Console
◦ # KERNEL: [CYCLE: 24] | PC: 21
◦ # KERNEL: Instruction: 00000000
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000001
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000019
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000016
◦ # KERNEL:   R31: 00000016

```

Console

```
◦ # KERNEL: [CYCLE: 25] | PC: 22
◦ # KERNEL: Instruction: 58000002
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000001
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000019
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000017
◦ # KERNEL:   R31: 00000016
```

Console

```
◦ # KERNEL: [CYCLE: 26] | PC: 24
◦ # KERNEL: Instruction: 00000000
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000001
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000019
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 00000019
◦ # KERNEL:   R31: 00000016
```

```

Console
◦ # KERNEL: [CYCLE: 27] | PC: 25
◦ # KERNEL: Instruction: program is done
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000001
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000019
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 0000001a
◦ # KERNEL:   R31: 00000016

```

Console

```
◦ # KERNEL: [CYCLE: 28] | PC: 26
◦ # KERNEL: Instruction: program is done
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000001
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000019
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 0000001b
◦ # KERNEL:   R31: 00000016
```

```

Console
◦ # KERNEL: [CYCLE: 29] | PC: 27
◦ # KERNEL: Instruction: program is done
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000001
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000019
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 0000001c
◦ # KERNEL:   R31: 00000016

```

```

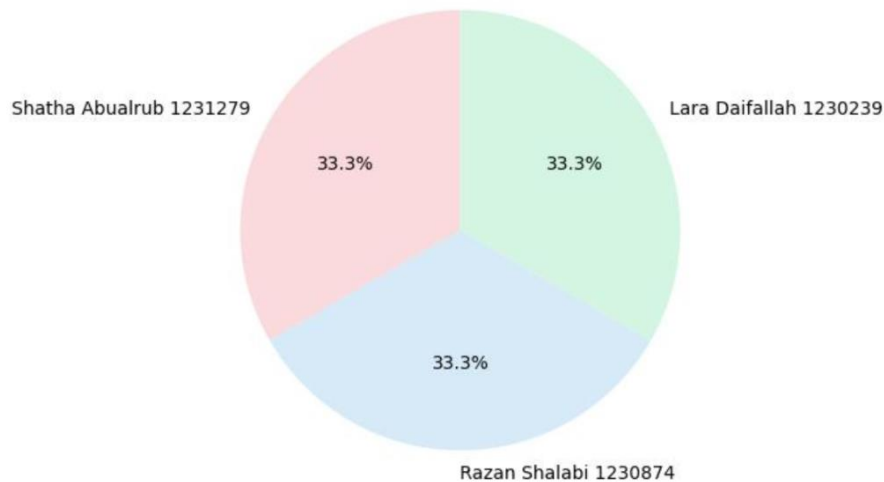
Console
◦ # KERNEL: [CYCLE: 30] | PC: 28
◦ # KERNEL: Instruction: program is done
◦ # KERNEL: Data Memory:
◦ # KERNEL:   Mem[15]: 00000000
◦ # KERNEL:   Mem[16]: 00000000
◦ # KERNEL:   Mem[17]: 00000000
◦ # KERNEL:   Mem[18]: 00000000
◦ # KERNEL:   Mem[19]: 00000000
◦ # KERNEL:   Mem[20]: 00000000
◦ # KERNEL: Registers:
◦ # KERNEL:   R00: 00000000
◦ # KERNEL:   R01: 0000000a
◦ # KERNEL:   R02: 0000000f
◦ # KERNEL:   R03: 0000000f
◦ # KERNEL:   R04: 000000ff
◦ # KERNEL:   R05: ffffffff00
◦ # KERNEL:   R06: 000000f0
◦ # KERNEL:   R07: 00000000
◦ # KERNEL:   R08: 0000000f
◦ # KERNEL:   R09: ffffffff5
◦ # KERNEL:   R10: 00000000
◦ # KERNEL:   R11: 00000000
◦ # KERNEL:   R12: 00000000
◦ # KERNEL:   R13: 00000001
◦ # KERNEL:   R14: 00000000
◦ # KERNEL:   R15: 00000019
◦ # KERNEL:   R16: 00000000
◦ # KERNEL:   R17: 00000000
◦ # KERNEL:   R18: 00000000
◦ # KERNEL:   R19: 00000000
◦ # KERNEL:   R20: 00000000
◦ # KERNEL:   R21: 00000000
◦ # KERNEL:   R22: 00000000
◦ # KERNEL:   R23: 00000000
◦ # KERNEL:   R24: 00000000
◦ # KERNEL:   R25: 00000000
◦ # KERNEL:   R26: 00000000
◦ # KERNEL:   R27: 00000000
◦ # KERNEL:   R28: 00000000
◦ # KERNEL:   R29: 00000000
◦ # KERNEL:   R30: 0000001d
◦ # KERNEL:   R31: 00000016

```

- All actual results met our expected results

Team Work

This project was completed through effective teamwork and close collaboration among all group members. The workload was carefully divided to ensure that each member contributed meaningfully to the overall design and implementation. Responsibilities were distributed across different aspects of the project, including datapath design, control logic implementation, module development, testing, simulation, and report writing. Regular discussions were held to review progress, resolve design issues, and ensure consistency between individual components. This collaborative approach allowed the team to integrate all parts smoothly and deliver a complete and well-functioning pipelined processor.



Conclusion

The design and verification of the 32-bit Simplified Predicated RISC Processor were successfully completed in full compliance with the architectural specifications defined in the project requirements. The processor was implemented using a five-stage pipeline consisting of Instruction Fetch, Instruction Decode, Execute, Memory Access, and Write-Back stages. This organization enabled efficient instruction overlap and high throughput while preserving correct execution in the presence of data, memory, and control dependencies.

A key outcome of this project was the effective integration of predicated execution within a pipelined architecture. By resolving predicate conditions early in the Instruction Decode stage and gating critical control signals such as RegWrite and MemWrite, predicated-false instructions were safely neutralized without requiring pipeline flushes. This early-resolution approach significantly reduced control overhead and preserved a fast and streamlined Execution stage, demonstrating an efficient balance between correctness and performance.

The processor's reliability was further strengthened through a comprehensive hazard management strategy. Data hazards were mitigated using an extensive forwarding network that supported the Rs, Rt, and predicate register Rp paths, allowing dependent instructions to execute back-to-back without unnecessary stalls. Load-use hazards were correctly detected by a dedicated Stall Unit, which temporarily paused the pipeline to ensure data correctness during memory-to-execution transitions. Control hazards were handled using a Kill Unit that injected NOP instructions on taken control-flow changes, limiting the branch penalty to a single clock cycle.

The architectural choices regarding control flow were validated through simulation. Implementing the Program Counter as register R30 and the return address as register R31 functioned correctly during CALL and JR operations, confirming proper procedure call and return behavior. Extensive simulation using a diverse instruction sequence demonstrated correct final register and memory states, consistent with theoretical expectations. The measured performance showed an efficient average CPI of approximately 1.15, reflecting effective pipeline utilization.